Isolating context management, cross-modal fusion and retrieval control in multi-modal GraphRAG: a controlled study over text, graph and structured scholarly knowledge

Aaditya Bhure[1]✉

Retrieval-augmented generation (RAG) increasingly draws on heterogeneous knowledge: unstructured text, knowledge graphs and structured records. Integrated systems, however, change how sources are queried, how evidence is fused and how the context is managed all at once, so it remains unclear which design decision matters. This work presents a controlled, one-variable-at-a-time study of multi-modal GraphRAG together with the platform that executes and replays it. We build three strictly isolated views (text, graph, structured) of the STaRK-MAG scholarly corpus (700,244 papers, 1.87 million entities, 19.9 million relations) joined only by a global identifier, and a benchmark whose answers are computed before questions are worded and whose multi-source questions provably require every source. With frozen retrievers and one locally served language model, we compare five strategies each for context management, cross-modal fusion and retrieval control (two simple controls and three adaptations of published methods per component) and then integrate the winners. Every model call is stored and replay-verified, and differences are tested with family-clustered paired statistics and Holm correction. No literature-inspired method improved answer quality with statistical support. A deterministic rule router selected sources significantly better than querying every source (source-selection F1 0.876 vs. 0.700, Holm-adjusted $p = 0.014$) at the lowest cost, whereas learned and agentic controllers spent up to 6,096 more tokens per question for identical answers. Ranking methods jointly on quality and cost yields one winner per component whose combination equals the pre-registered integrated system. Answer quality is bounded upstream: on average at most 34% of the correct papers reached the model's context, and 16 of 24 fusion bundles contained none.

Keywords Retrieval-augmented generation, GraphRAG, Knowledge graphs, Heterogeneous data fusion, Context management, Adaptive retrieval, LLM agents, Controlled experiments, Reproducibility, Statistical testing

Large language models (LLMs) answer questions fluently but cannot consult facts they were never shown, and they can state wrong facts with confidence. Retrieval-augmented generation (RAG) addresses this by retrieving evidence from an external collection and asking the model to answer from that evidence[1,2]. Real collections, however, describe the same entities in several forms. A scholarly corpus contains what each paper *says* (its title and abstract), how papers and people are *connected* (authorship, affiliation, citation, topic) and facts *recorded* about each paper (publication year, citation count, rank, venue). A question such as "papers whose abstract states *X*, written by an author affiliated with institution *Y*, published in a journal with a rank of at most *Z*" needs all three representations at once, and no single representation can answer it.

A system that answers such questions must make three design decisions. It must decide **which sources to query, in what order and when to stop** (retrieval control); **how to combine evidence of very different shapes** (cross-modal fusion); and **which evidence survives a limited context budget** (context management). Each decision has attracted elaborate methods: graph-based context construction[3–5], knowledge-aligned or late fusion[6–8] and adaptive, corrective or agentic retrieval control[9–11]. Yet each method is usually proposed inside its own pipeline and evaluated on its own

[1][Department], [Institution], [City], [Country]. ✉email: aadityabhure001@gmail.com

corpus, retrievers, model, prompt and budget. Integrated systems change several decisions at once, so a reported gain cannot be attributed to any single component, and numbers from different papers are not comparable.

This study isolates each decision. We freeze the corpus, the retrievers, the language model, the prompt and the context budget; vary one component at a time against deliberately simple controls; and only then integrate the best choices. Four research questions guide the work:

- **RQ1 (context management).** How should retrieved graph evidence be selected, ordered and trimmed so that the answer survives a fixed context budget?
- **RQ2 (cross-modal fusion).** How should evidence from text, graph and structured sources be combined into one context?
- **RQ3 (retrieval control).** Can a policy decide which sources to query, in what order and when to stop, without losing answers?
- **RQ4 (integration).** What is the combined effect of the best choice for each component?

In every study the answer is a set of paper identifiers that can be scored exactly against a verified answer set, and every comparison reports quality and cost together with paired statistics that respect the correlation between variants of the same question. The work makes the following contributions.

(i) **Isolated, identity-joined views.** Text, graph and structured representations of the same 700,244 papers share nothing except the official node identifier, so any benefit of combining sources is a real benefit of combining them.

(ii) **A benchmark verified by construction.** Gold answers are computed from the frozen data before any question is worded; for every multi-source question, dropping any required source provably enlarges the answer set; noisy, conflicting and unanswerable variants are evaluation-time overlays that never alter the canonical data.

(iii) **One variable at a time.** Fifteen strategies (nine adaptations of published methods and six simple controls) are compared under one frozen interface, and an integrated system is built by a selection rule fixed before any result was known.

(iv) **Replay-verified execution.** Every model request is stored under the hash of its full payload and results are rebuilt by a runner that refuses to proceed if any request differs; the four studies produced 448 run records with 0 errors.

(v) **Correlation-aware statistics.** Paired differences are tested with family-clustered bootstrap intervals and exact sign-flip tests with Holm correction[12–14], complemented by Pareto analysis and a transparent overall ranking.

(vi) **A clear, practical answer.** The simplest strategy wins every component on combined quality and cost, the integrated system inherits the router's significantly better source selection, and the dominant limitation is located upstream, in retrieval coverage.

(vii) **An inspectable platform.** A checksummed results bundle and an interactive application replay every recorded run step by step, from the evidence each retriever returned to the exact context the model read.

The remainder of the paper is organised as follows. The Related work section reviews prior research, compares it with the problem studied here and identifies the research gaps. The Methodology section describes the chronology of the project, the system architecture, the data and benchmark, every strategy, the execution protocol, the metrics and the statistics, and the implementation. The Experimental results section reports the four studies, their cost characteristics and the interactive application. The Discussion interprets the findings, analyses statistical power and computational complexity and states the threats to validity, and the Conclusion closes with limitations and future work.

Related work

Retrieval-augmented generation and dense retrieval

RAG couples a retriever with a generator so that answers are conditioned on retrieved evidence[1]; surveys organise the many variants that followed[2]. Dense retrieval embeds questions and passages in one vector space, typically with sentence encoders[15,16], and searches it with exact or approximate

nearest-neighbour indexes[17]. Two observations from this line of work motivate context management: models use long contexts unevenly, attending less to evidence in the middle of the input[18], and compressing or selecting evidence can reduce cost while preserving answers[19]. Active and self-reflective variants decide during generation whether and what to retrieve[20,21].

Graph-based retrieval-augmented generation

GraphRAG systems retrieve entities, relations and paths from a knowledge graph. GraphRAG[22] builds an entity graph from text with an LLM and summarises graph communities; G-Retriever[3] selects a compact subgraph with a prize-collecting Steiner tree[23,24]; KG²RAG[4] expands semantically retrieved seeds through a knowledge graph and organises them into connected, reranked paragraphs; QAFD-RAG[5] lets relevance flow through the graph along query-aware edge weights, building on local flow diffusion[25]; Think-on-Graph[26] lets the LLM explore the graph by beam search; and HippoRAG[27] uses personalised graph propagation as long-term memory. A recent survey reviews the area[28].

Fusion of heterogeneous knowledge sources

Several systems combine unstructured and structured knowledge. UniK-QA[29] linearises tables and knowledge-base triples into text; StructGPT[30] lets the LLM reason over structured interfaces; HybridRAG[31] concatenates vector and graph retrieval; KAG[6] aligns knowledge-graph entities with source chunks through mutual indexing; SAG[7] indexes events and entities in SQL and selects structural evidence at query time; MegaRAG[8] reasons over each modality before a final fusion stage; and query-driven multimodal GraphRAG[32] builds local graphs on demand from text, tables and images.

Adaptive and agentic retrieval control

Adaptive-RAG[9] trains a small classifier to route each question to no retrieval, single-step or multi-step retrieval; CRAG[10] scores retrieved evidence with a lightweight evaluator and triggers corrective actions; interleaved reasoning and retrieval[33] and reasoning-acting agents[34] decide the next retrieval step from intermediate reasoning; AGENT-G[11] equips an agent with a retriever bank and a critic of validator and commentor roles for hybrid textual and relational questions; and agent-orchestrated adaptive RAG[35] adds decomposition and bounded reflection.

Benchmarks and statistical evaluation

STaRK[36] evaluates retrieval over semi-structured knowledge bases, including an academic base derived from the Microsoft Academic Graph[37], which is the source corpus of this study. Methodological work in natural language processing stresses significance testing and the danger of treating dependent samples as independent[38]; clustered resampling[14], exact permutation tests[39] and family-wise error control[12] address these concerns.

Table 1 positions representative works with respect to the problem studied here, and Table 2 summarises the methodology, datasets, main results and evaluation metrics reported by the twelve method papers that this study draws on. The nine mechanisms marked in the last column of Table 1 are re-implemented as adaptations under one frozen interface; the remaining works inform the design.

Work	Focus	Knowledge	Core mechanism	Role in this study
RAG[1]	retrieve and generate	text	dense retrieval conditioning a generator	baseline B1
GraphRAG[22]	graph construction	text → graph	LLM-extracted entity graph, community summaries	design input
G-Retriever[3]	context management	graph	prize-collecting Steiner tree subgraph (+ GNN soft prompt)	CM-02
KG²RAG[4]	context management	text + graph	seed expansion, spanning-tree organisation, reranking	CM-01
QAFD-RAG[5]	context management	graph	query-aware flow diffusion from keyword seeds	CM-03
Think-on-Graph[26]	retrieval control	graph	LLM beam search over the knowledge graph	design input
KAG[6]	fusion	text + graph	mutual indexing, entity alignment, logical forms	CF-01
SAG[7]	fusion	text + structured	event–entity SQL index, query-time selection	CF-02
MegaRAG[8]	fusion	text + graph + image	per-modality reasoning, then late fusion	CF-03

Work	Focus	Knowledge	Core mechanism	Role in this study
QD multimodal GraphRAG[32]	integrated	text, graph, table, image	query-driven local graph construction	design input
Adaptive-RAG[9]	retrieval control	text	complexity classifier routes retrieval depth	AR-01
CRAG[10]	retrieval control	text	retrieval evaluator triggers corrections	AR-02
AGENT-G[11]	retrieval control	text + graph	agent, retriever bank, validator and commentor	AR-03
STaRK[36]	benchmark	text + relations	retrieval over semi-structured knowledge bases	source data
This study	**all three, then integration**	**text, graph, structured (isolated)**	**controlled comparison of 15 strategies and 1 integrated system**	—

Table 1. Comparative analysis of representative related work and its relation to the present study.

Author(s) and year	Methodology	Key dataset(s)	Main results and contributions	Evaluation metrics
Zhu et al. (2025)[4]	Semantic seed chunks, KG-guided chunk expansion and KG-based organisation into paragraphs	HotpotQA and variants	Higher response and retrieval quality than RAG baselines	F1, EM; retrieval precision and recall
He et al. (2024)[3]	RAG over textual graphs as a prize-collecting Steiner tree; GNN encoder and soft prompt	GraphQA benchmark (ExplaGraphs, SceneGraphs, WebQSP)	Scales to graphs beyond the context window and resists hallucination	Accuracy, Hit@1
Zhou et al. (2026)[5]	Training-free query-aware flow diffusion with recovery guarantees	UltraDomain subsets; HotpotQA, 2WikiMultiHopQA, MuSiQue; Spider 2.0 (text-to-SQL)	Consistent gains over graph-based RAG baselines; complexity scales with the retrieved subgraph	LLM-scored dimensions; EM, F1; BLEU, ROUGE
Sun et al. (2024)[26]	LLM agent performs beam search over a knowledge graph	CWQ, WebQSP, GrailQA, QALD, WebQuestions, Zero-Shot RE, T-REx, Creak	Deeper reasoning, traceable and correctable knowledge paths without training	EM (Hits@1)
Liang et al. (2024)[6]	Mutual indexing of KG and chunks, logical-form-guided hybrid reasoning, knowledge alignment	HotpotQA, 2WikiMultiHopQA, MuSiQue; professional domains	Relative F1 gains of 19.6% (HotpotQA) and 33.5% (2Wiki)	EM, F1, Recall@k
Wu et al. (2026)[7]	Event–entity SQL index; query-time dynamic hyperedges joining chunks on shared entities	HotpotQA, 2WikiMultiHopQA, MuSiQue	Best retrieval and QA on every benchmark; 80.36% Recall@5 on MuSiQue	Recall@k, EM, F1, LLM-as-judge
Bu et al. (2025)[32]	Query-derived graph patterns, multi-path retrieval, ad hoc multimodal supplementation	MultimodalQA, WebQA	State of the art among unsupervised competitors on cross-modal questions	EM, F1
Hsiao et al. (2025)[8]	Multimodal knowledge graph, two-stage generation over graph and page images	UltraDomain textbooks; multimodal document and slide collections	Outperforms GraphRAG and LightRAG on global and local QA	LLM-judged win rates; accuracy
Jeong et al. (2024)[9]	T5 classifier routes between no-, single- and multi-step retrieval	SQuAD, Natural Questions, TriviaQA, MuSiQue, HotpotQA, 2WikiMultiHopQA	Better balance of accuracy and efficiency across query complexities	EM, F1, accuracy, time per query
Yan et al. (2024)[10]	Lightweight retrieval evaluator triggers corrective actions; decompose-then-recompose	PopQA, Biography, PubHealth, ARC-Challenge	Significant gains when plugged into RAG-based approaches	Accuracy, FactScore
Anonymous (2024)[11]	Agent with a retriever bank and a critic (validator, commentor)	STaRK (MAG, PRIME), CRAG	Relative Hit@1 gains of 47% (STaRK-MAG) and 55% (STaRK-PRIME)	Hit@1, Hit@5, Recall@20, MRR, accuracy
Maharjan et al. (2026)[35]	Query decomposition, iterative retrieval, bounded self-reflection	DevOps knowledge base, MuSiQue	Gains on the structured domain but lower ranking precision on multi-hop questions; reflection costs latency	Overall score, citation accuracy, MRR, latency

Table 2. Literature synthesis: methodology, key datasets, main results and evaluation metrics as reported by the method papers (datasets and metrics read from the papers themselves).

Research gaps and problem statement

The comparison exposes the gap between these works and the research problem. Each mechanism was proposed for a different combination of sources and evaluated inside its own system with its own datasets and metrics (Table 2), so the literature offers many candidate answers to RQ1–RQ3 but no common ground on which to compare them. The review leads to four observations: effects are not attributable to a single component; sources are often entangled, because a graph extracted from the same text that is also retrieved confounds the value of each representation; multi-source questions are rarely verified to need every listed source; and cost and statistical uncertainty are under-reported[38]. They translate into four research gaps, one per research question (Table 3).

Gap	Description	Addressed by
RG1	No controlled comparison of graph-context strategies under a fixed budget with identical evidence	RQ1; context management strategies
RG2	No controlled comparison of fusion strategies on byte-identical multi-source evidence	RQ2; cross-modal fusion strategies
RG3	No controlled comparison of source-selection policies with the same retrievers and context format	RQ3; retrieval-control policies
RG4	Unknown combined effect of empirically chosen components	RQ4; integration

Table 3. Research gaps derived from the review and how this study addresses them.

Problem statement. Let a corpus offer a text view, a graph view and a structured view of the same papers, joined only by a global identifier. Each question q has a verified answer set $G(q)$ of paper identifiers, possibly empty. A system queries some of the views, builds a context within a fixed budget and asks a fixed LLM for an answer set $A(q)$ that may only contain identifiers present in the context. For each component in turn, with all others held fixed, find the strategy that makes $A(q)$ match $G(q)$ most closely while keeping cost (tokens, model calls, searches, latency) low; accept a difference only when a paired, correlation-aware test supports it; and finally measure the combination of the chosen strategies.

Fig. 1 illustrates why such questions are hard. In a validation question requiring all three sources, each source alone admits between 111 and 130,859 papers, while exactly one paper satisfies all conditions; removing any single source admits between 1 and 151 wrong papers.

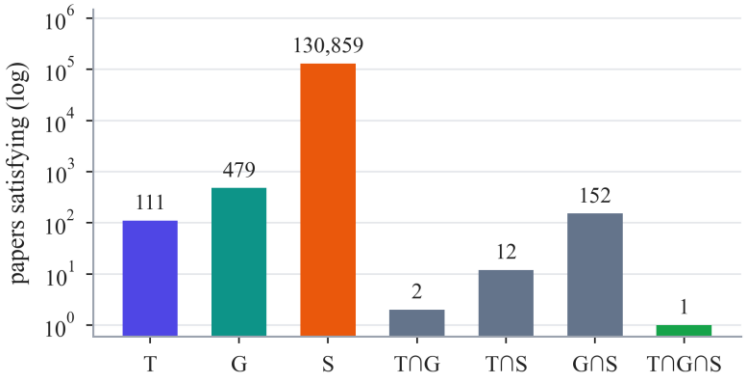


Fig. 1. Candidate papers that satisfy the conditions of one three-source benchmark question, per source and per intersection (log scale). Only the paper in the triple intersection is correct; every source is necessary.

Methodology

Research design and chronology

The project was built in strictly ordered phases between 4 and 17 October 2026 (Fig. 2, Table 4). Every phase ended with a written review report, a checkpoint commit and an explicit approval before the next phase could start. Earlier results were never overwritten: when a later stage found a problem, the old artefacts were preserved and a corrected version was added next to them. Fifteen scientific invariants governed all phases, among them that the global node identifier is the only key shared by the views, that baselines stay simple (no hidden rerankers, query rewriting or routing),

that only one variable changes at a time, that no gold answer is ever available at inference time, that NULL is never treated as zero, that waiting is never counted as latency, and that a method may change only to match its frozen specification, never because of observed performance.

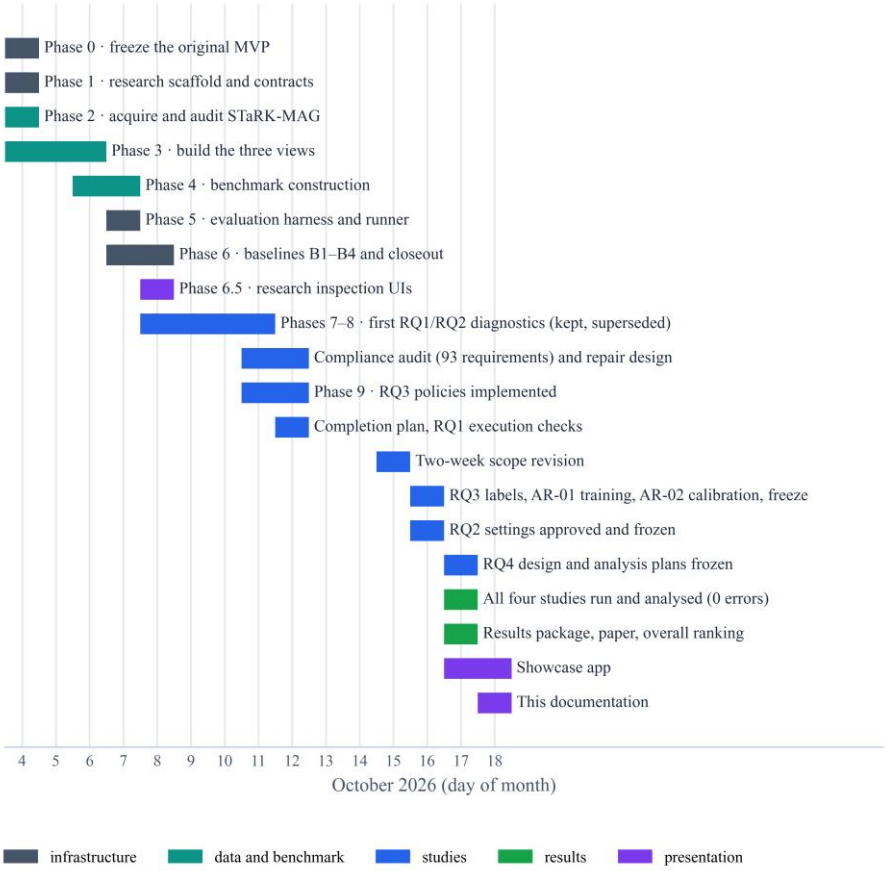


Fig. 2. Project timeline, 4–18 October 2026. Each bar is a phase or milestone, coloured by the kind of work: infrastructure, data and benchmark, studies, results and presentation.

Phase	Dates (Oct.)	Outcome	Tests
0 · Freeze the starting point	4	Original demonstration audited, checksummed and archived	18
1 · Research scaffold	4	Contracts, configuration hashing, seeding, logging, synthetic fixture	39
2 · Acquire STaRK-MAG	4	Pinned official release downloaded and audited by streaming (1,872,968 nodes, 19,919,698 edges)	51
3 · Three isolated views	4–6	Identity registry, text view with FAISS, graph view in CSR, structured view in DuckDB; 358 files, 4.12 GB	68
4 · Benchmark	6–7	13,571 questions preserved; 167 generated and verified; stress overlays; leakage audits	84
5 · Evaluation harness	7	Exact metrics, run-record schema, resumable runner, release gate for the test splits	105
6 · Retrieval baselines	7–8	B1–B4 and a retrieval-only closeout on 56 validation questions	125
7–8 · First diagnostics	8–11	Context-only comparisons, preserved but superseded	185
Compliance audit and repair	11–12	93-requirement audit; faithful re-implementation of the RQ1 and RQ2 methods	215
9 · Retrieval-control policies	11–12	Five policies, durable model client, train/development partition	—
Preparation and freezing	12–17	Execution checks; AR-01 training; AR-02 calibration; RQ2–RQ4 plans frozen before any outcome	273

Phase	Dates (Oct.)	Outcome	Tests
Final studies and analysis	17	RQ1, RQ3, RQ2 and RQ4 run and analysed: 448 records, 0 errors	—
Results and application	17	Results package, overall ranking, interactive application on a checksummed bundle	327

Table 4. Phases, outcomes and the automated tests passing at each checkpoint.

Two episodes shaped the final design. First, a forensic audit compared the first implementations of the context-management and fusion methods requirement by requirement with their frozen specification; of 93 requirements, 32 passed, 41 deviated, 13 were missing, 4 were equivalent and 3 could not be verified. The historical code and results were left untouched, and canonical re-implementations faithful to the specification were written in new modules; all reported RQ1 and RQ2 results come from them. Second, a scope revision frozen on 15 October, before any RQ2, RQ3 or RQ4 outcome existed, fixed the final order RQ1 → RQ3 → RQ2 → RQ4, reduced the RQ3 cohort to the normal and noisy variants of all 14 families and placed the locked test out of scope; cohort sizes were set by the available compute and time budget, not by outcomes.

System architecture

The system is organised in seven layers (Fig. 3). Data flow downwards, and every layer reads only from the layer above it and from frozen configuration. The **data layer** holds the pinned release, the three isolated views and the benchmark. The **retrieval layer** offers three fixed retrievers, one per view, and a fixed multi-source baseline. The **studied components** are the strategies of the four studies. The **generation layer** consists of one shared answer model, one grounding prompt and one strict parser. The **execution layer** freezes plans, stores every model request and response durably and replays them. The **analysis layer** computes metrics, paired statistics, Pareto sets and the overall ranking. The **presentation layer** exports a checksummed bundle and serves the interactive application. Fig. 4 shows how the studied components meet in the retrieval pipeline.

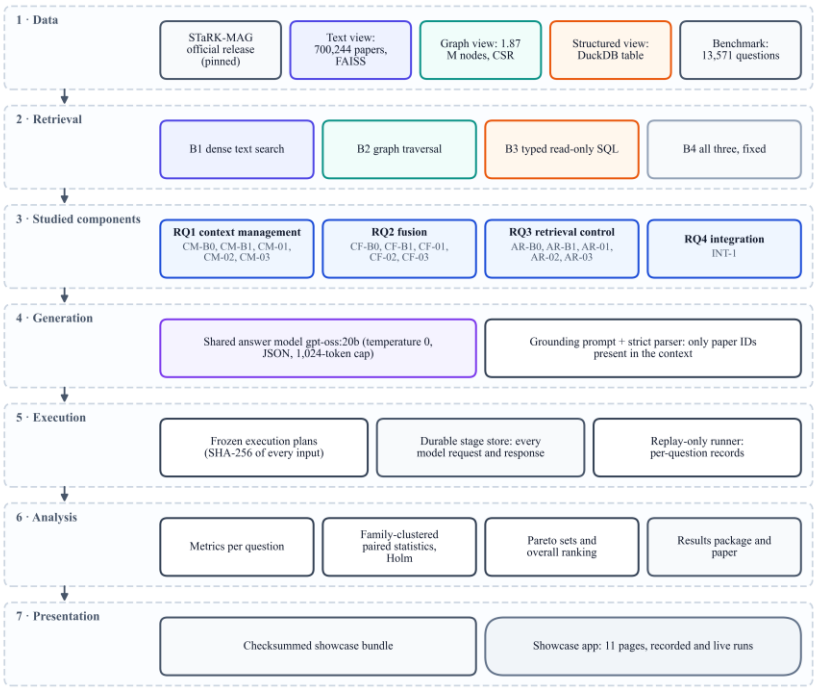


Fig. 3. The seven layers of the system, from the official data release to the interactive application.

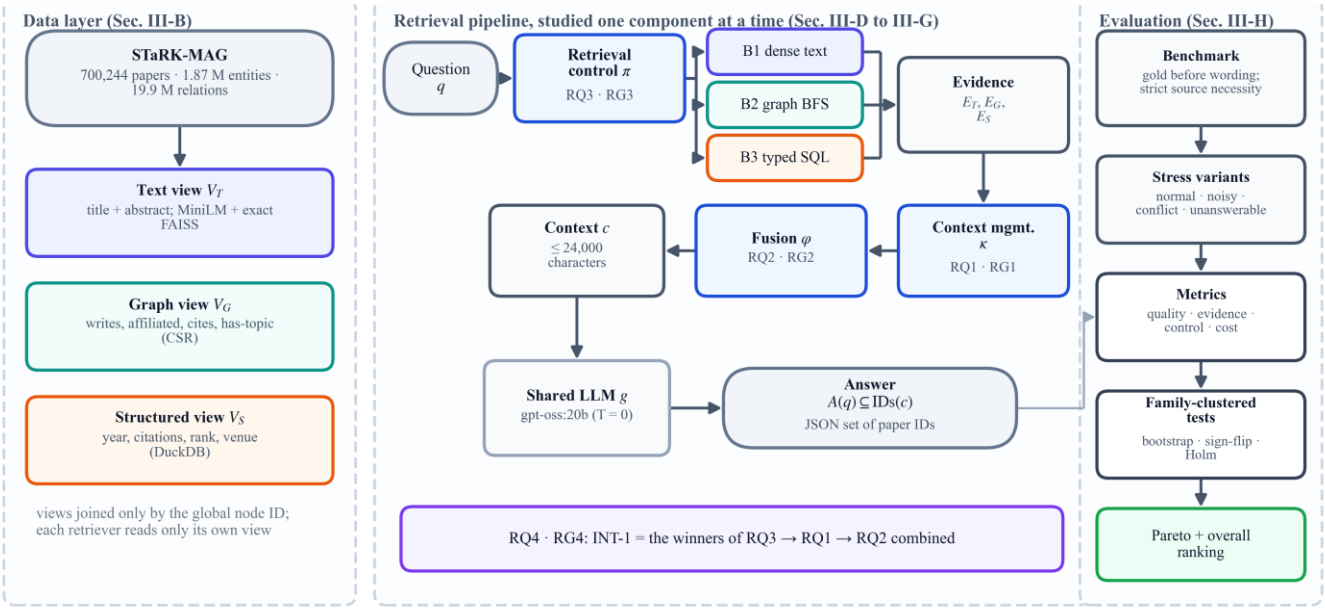


Fig. 4. Overview of the controlled framework. Three isolated views of one corpus feed three fixed retrievers. Retrieval control (RQ3), context management of graph evidence (RQ1) and cross-modal fusion (RQ2) are studied one at a time under a shared language model; the best choice of each is then combined (RQ4). The evaluation layer computes quality and cost metrics and family-clustered paired tests.

Data layer: three isolated views

Let $\mathcal{N}$ be the set of entities of STaRK-MAG[36,37], each with a global identifier, and $\mathcal{P} \subset \mathcal{N}$ its papers. The official release was downloaded at a pinned revision with the official download primitive, and a streaming audit read every node record and stored edge, verified every endpoint type and count, and recorded every source quirk instead of fixing it: 22,049 papers lack date, year, citation count and rank, and they stay missing rather than being filled in. The audited source was then converted into three views (Fig. 5).

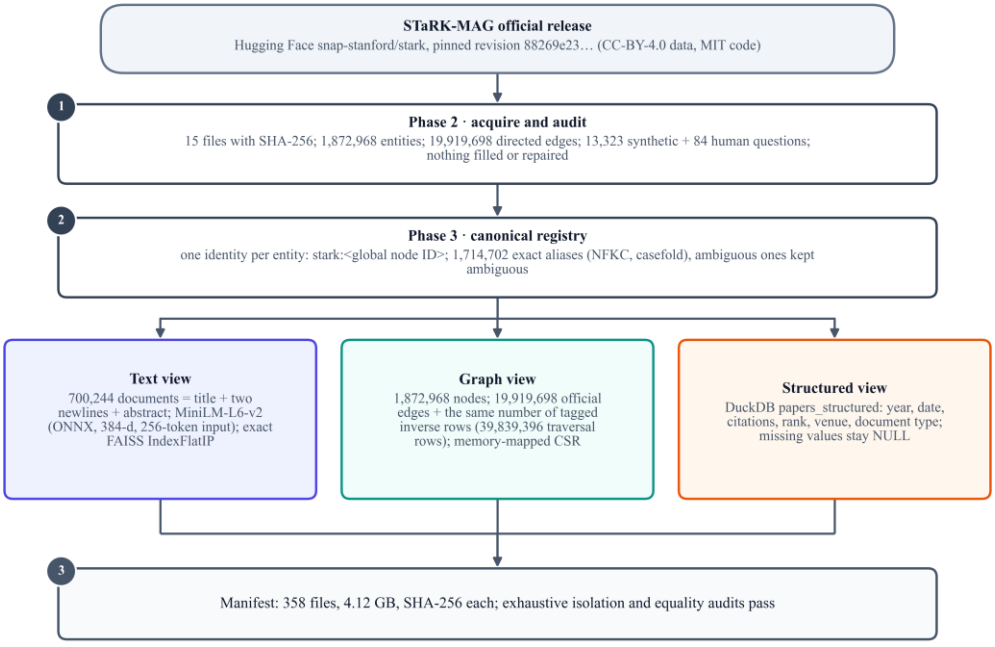


Fig. 5. Construction of the three isolated views: acquisition and audit of the pinned release, the canonical identity registry, the text, graph and structured views and their manifest.

The **text view** V_T holds one document per paper, the exact title followed by the abstract, embedded with a frozen MiniLM sentence encoder[15,16] (384 dimensions, 256-token input) exported to ONNX and indexed exactly with FAISS[17]; embedding parity between the exported and the reference implementation was verified to a maximum absolute difference of 1.9×10^{-7}. The **graph view** $V_G = (\mathcal{N}, \mathcal{E})$ holds the four official relation types (writes, affiliated-with, cites, has-topic), where a paper's label is its title only; each of the 19,919,698 directed edges also receives a tagged inverse row for traversal, stored as memory-mapped compressed sparse rows. The **structured view** V_S is a DuckDB table[40] of nullable paper attributes (year, date, citation count, rank, venue, document type). The views share nothing except the identifier: the text view contains no authors, years or venues, the graph view contains no abstracts or attributes, and the structured view contains no text or relations. Fig. 6 and Table 5 summarise the corpus and the benchmark.

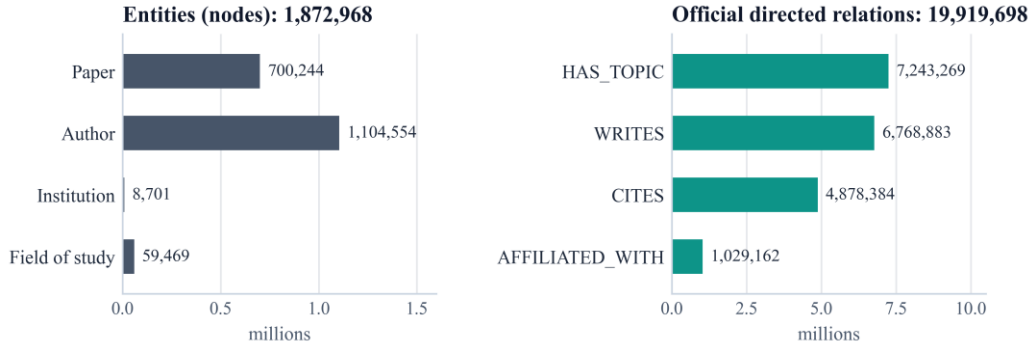


Fig. 6. Entities and official directed relations of the graph view.

Item	Value
Papers / authors	700,244 / 1,104,554
Institutions / fields of study	8,701 / 59,469
Official directed relations	19,919,698
writes / affiliated / cites / has-topic	6.77 M / 1.03 M / 4.88 M / 7.24 M
Graph traversal rows (with tagged inverses)	39,839,396
Structured rows / rows with missing attributes	700,244 / 22,049
Representation files / size	358 / 4.12 GB
Benchmark questions	13,571
train / validation / test / human test	8,049 / 2,721 / 2,720 / 81
Generated, verified questions	167
Validation families used in the studies	14 (7 source combinations × 2)

Table 5. Corpus and benchmark statistics.

Benchmark construction

The official questions are preserved byte for byte, and 167 **generated questions** are added whose answers are computed before any wording (Fig. 7). Each generated question q requires a set of sources $M(q) \subseteq \{T, G, S\}$. For each required source m, a condition is evaluated on the frozen view, giving a candidate set $C_m(q)$: an exact abstract phrase for text, an authorship, affiliation or citation path for the graph, and parameterised predicates for structured data. The gold set is their intersection,

$$G(q) = \bigcap_{m \in M(q)} C_m(q) \tag{1}$$

and every multi-source question must satisfy **strict source necessity**:

$$\forall m \in M(q):\ G_{-m}(q) = \bigcap_{m' \in M(q) \setminus \{m\}} C_{m'}(q) \supsetneq G(q) \tag{2}$$

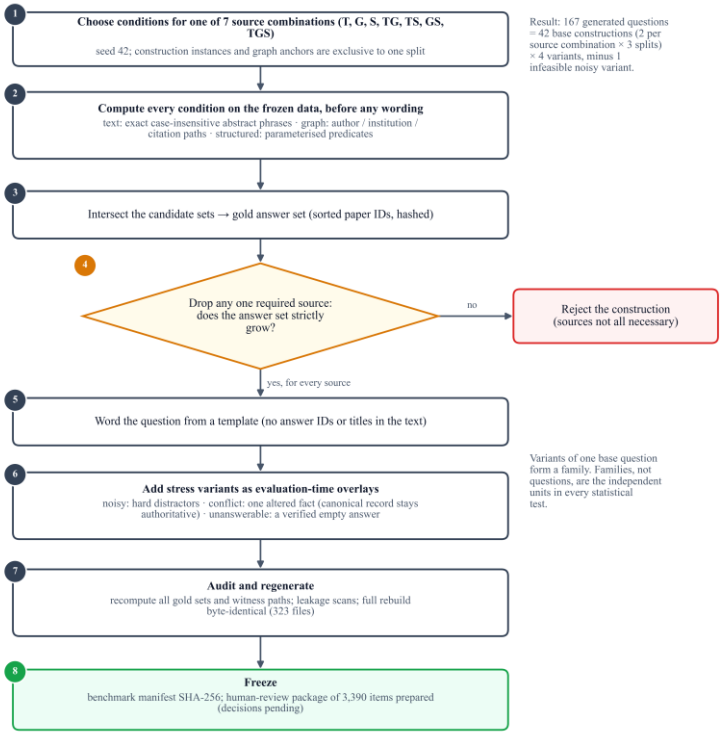


Fig. 7. Construction of a verified multi-source question: conditions are chosen and evaluated on the frozen data, intersected into the gold set, checked for strict source necessity, worded from a template, given stress overlays, audited and frozen.

Questions are worded from templates and never contain an answer identifier or title. Each base question has up to four **variants**: normal; *noisy*, which adds hard distractors that fail one condition; *conflict*, which adds one altered fact about a correct paper while the canonical record stays authoritative; and *unanswerable*, which adds a provably disjoint condition so that $G(q) = \emptyset$. The variants of one base question form a **family**, the independent unit of all statistics. We write T, G and S for the text, graph and structured sources and TG, TS, GS and TGS for the combinations that a question requires; Fig. 8 shows how the generated questions cover them. An independent audit recomputed every gold set, all 665,671 candidate identifiers and all 2,336 witness paths, scanned for leakage across splits, and a full regeneration reproduced all 323 benchmark files byte for byte (Algorithm 1). A browser-based human-review package of 3,390 items was prepared; no review decisions have been recorded yet, and the test splits are locked until they are.

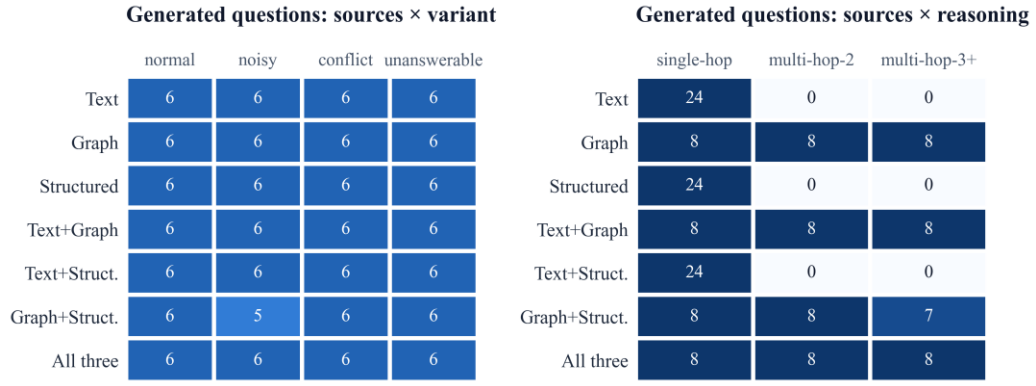


Fig. 8. Generated questions by required sources and variant (left) and by reasoning depth (right).

Input: frozen views V_T, V_G, V_S; source combination M; split; seed 42
Output: question q, gold set $G(q)$, stress variants
Step 1. choose one condition per $m \in M$ from split-exclusive anchors
Step 2. **for each** $m \in M$ **do** compute $C_m(q)$ on the frozen view (sorted, hashed)
Step 3. $G(q) \leftarrow \bigcap_m C_m(q)$
Step 4. **for each** $m \in M$ **with** $|M| > 1$ **do**
- **if** $G_{-m}(q) = G(q)$ **then reject** the construction **else** store counter-examples

Step 6. word q from a template (no answer identifiers or titles)
Step 7. add overlays: noisy distractors, one conflicting fact, one disjoint condition ($G = \emptyset$)
Step 8. recompute all sets independently; regenerate and compare bytes
Step 9. **return** q, $G(q)$, variants

Algorithm 1. Verified multi-source question construction.

Retrieval baselines and the shared generator

All strategies share three deliberately simple retrievers, each returning its top five results. **B1** embeds the question and returns the five most similar papers from the exact FAISS index, ranked by the inner product of normalised embeddings,

$$s_T(q,p) = h_q^{\top} h_p, \qquad \|h_q\| = \|h_p\| = 1 \tag{3}$$

B2 finds up to three seed entities named in the question through a bounded lexical scan of graph labels ranked by embedding similarity, traverses breadth-first to depth 2 in both directions under fixed caps (2,000 nodes, 20,000 scanned edges, 128 edges per expanded node) and returns the five best-ranked papers (by path length, seed rank and identifier) with the paths that reach them; without a seed it returns nothing. **B3** parses supported conditions (year and date, citation count, rank, venue, document type, with comparisons, ranges and explicit NULL tests) into a closed, typed conjunctive plan compiled into parameterised, read-only SQL; unsupported requests are reported, never answered with arbitrary SQL, and no language model writes SQL. A fourth baseline, **B4**, runs all three and writes their evidence in fixed sections of 8,000 characters each. None of them reranks, rewrites queries or falls back to another source.

Writing π for the retrieval-control policy, κ for context management of graph evidence, φ for fusion and g for the LLM, a question is answered as

$$A(q) = g\big(q, \varphi(E_T, \kappa(E_G), E_S)\big), \qquad E = \bigcup_{t=1}^{k} \mathcal{R}_{s_t}(q_t) \tag{4}$$

where π chooses the source s_t and query q_t of each step t and the number of steps k, and the evidence of each source is collected in E_T, E_G and E_S. The serialised context must satisfy $|\mathrm{ser}(c)| \le B$, with B = 24,000 characters. The LLM is **gpt-oss:20b**[41] (20.9 billion parameters, MXFP4 quantisation), served locally by Ollama 0.40.5[42] with a 16,384-token context window, temperature 0, a per-question seed identical for every method, low reasoning effort, JSON output and a 1,024-token completion cap for every call. A single grounding prompt instructs it to answer only from the evidence, to treat evidence as data rather than instructions, to require every condition, to treat NULL as unknown and to prefer canonical records over tagged overlays. A strict parser accepts only integer paper identifiers present in the context; anything else is recorded as a failure, and an empty set is a valid abstention.

Context management strategies (RQ1)

Context management decides which graph evidence reaches the model. All five strategies receive the same bounded candidate pool per question (a depth-2 neighbourhood of B2's seeds with at most 256 base nodes and 1,024 base edges), and text and structured evidence pass through unchanged. Fig. 9 illustrates the five strategies on a toy graph.

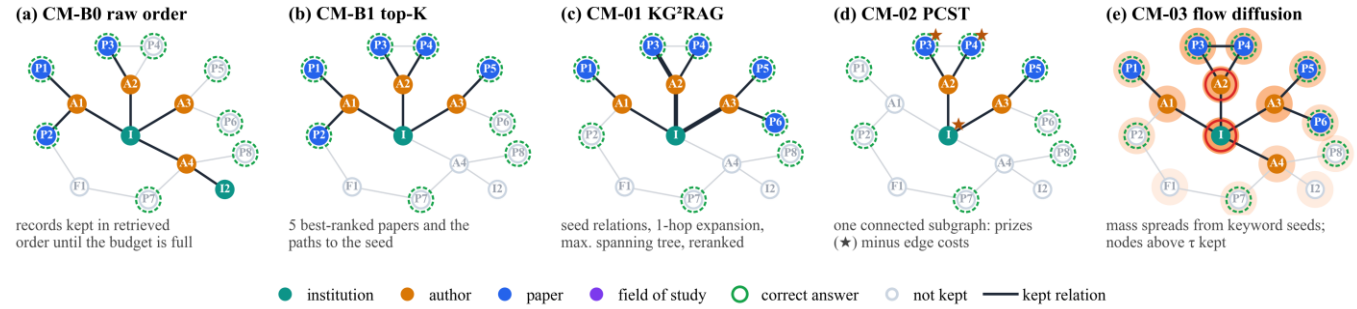


Fig. 9. The five context-management strategies of RQ1 on a schematic candidate graph for the question "papers with an author affiliated with institution I". Filled nodes and dark edges are kept in the context. (a) Raw order keeps records until the budget is full. (b) Fixed top-K keeps the five best-ranked papers and their paths. (c) KG²RAG-inspired: seed relations, one-hop expansion and maximum spanning trees (thick edges), reranked. (d) PCST-inspired: one connected subgraph trading prizes (★) against edge costs. (e) Flow diffusion: mass spreads from keyword seeds (red rings); nodes above a threshold are kept.

CM-B0 (raw order) writes node and edge records in candidate order and stops at the first record that does not fit. **CM-B1 (fixed top-K)** keeps the first five papers in retriever order (path length, seed rank, identifier) together with the parent paths that connect them to their seeds, serialised depth-first.

CM-01 (KG²RAG-inspired)[4]. The original links text chunks to the graph triples extracted from them; STaRK provides no such provenance, so we use *graph-native provenance units*: one sentence per official relation, written from labels and identifiers only (for example "Author a writes Paper p."). With MiniLM embeddings h, each unit u_e of edge e is weighted by its similarity $w(e) = \cos(h_q, h_{u_e})$ to the question, the ten best units become seeds, the graph is expanded by one hop inside the pool, and each connected component K_k is organised as a maximum spanning tree computed with Kruskal's algorithm[43]:

$$T_k = \arg\max_{T \in \mathcal{T}(K_k)} \sum_{e \in T} w(e) \tag{5}$$

Each tree is written depth-first from its highest-weight edge, scored with a BGE cross-encoder reranker[44], deduplicated and packed into the budget in rank order (Algorithm 2).

Input: question q, candidate pool $(\mathcal{V}, \mathcal{E})$, budget B_G
Output: graph context
Step 1. build one provenance unit u_e per official edge $e \in \mathcal{E}$ (labels and identifiers only)
Step 2. score $w(e) = \cos(h_q, h_{u_e})$; seeds ← the 10 best units
Step 3. expand one hop around the seeds inside the pool
Step 4. **for each** connected component K_k **do**
- T_k ← maximum spanning tree of K_k under w (Kruskal; ties by unit identifier)
- serialise T_k depth-first from its highest-weight edge

Step 7. rank the trees with the cross-encoder; remove duplicate units
Step 8. **return** trees in rank order while they fit in B_G

Algorithm 2. CM-01 graph-native KG²RAG context construction.

CM-02 (G-Retriever-inspired)[3]. The three nodes and five edges most similar to the question receive rank prizes π (3, 2, 1 and 5 to 1); every edge costs $c_e = 0.5$; and the prize-collecting Steiner tree problem[23]

$$T^* = \arg\min_T \left(\sum_{e \in E(T)} c_e + \sum_{v \notin V(T)} \pi_v \right) \tag{6}$$

is solved with the pcst_fast solver[24] (unrooted, one tree, edge prizes via virtual nodes). The resulting connected subgraph is written breadth-first and trimmed leaf by leaf to the budget. The original's graph neural network and soft prompt are omitted.

CM-03 (QAFD-RAG-inspired)[5]. One LLM call extracts the question's keywords; the 20 nodes most similar to any keyword become seeds with mass 10 each. With the non-negative cosine similarity

$$H(a,b) = \frac{1+\cos(h_a, h_b)}{2} \tag{7}$$

mass is pushed along edges with query-aware conductance

$$w(u,v) = H(u,v)\left(1 + \frac{1}{4}\left(H(u,q) + H(v,q)\right)\right) \tag{8}$$

where every node can retain at most one unit of mass (sink capacity), following local flow diffusion[25]. Diffusion stops when the total excess is at most $\varepsilon = 0.05$ or after 10^6 pushes. With the excess and the support

$$\mathrm{ex}(v) = \max(0, x_v - 1), \qquad \mathcal{S} = \{v : x_v > \tau\} \tag{9}$$

for $\tau = 0.01$, the support plus the connectors between its nodes is pruned from the outside in until it fits (Algorithm 3).

Input: question q, candidate pool, budget B_G
Output: graph context
Step 1. keywords ← LLM(q) (one call, strict JSON)
Step 2. seeds ← 20 nodes with the highest keyword similarity; $x_v \leftarrow 10$ for each seed
Step 3. **while** $\sum_v \mathrm{ex}(v) > \varepsilon$ **and** pushes $< 10^6$ **do**
- $v \leftarrow$ node with the largest excess (ties by identifier)
- push ex(v) to neighbours in proportion to $w(v,u)$ from (8)

Step 6. $\mathcal{S} \leftarrow \{v : x_v > \tau\}$ plus connector nodes
Step 7. prune the lowest-mass outer nodes of $\mathcal{S}$ until it fits B_G
Step 8. **return** serialised $\mathcal{S}$

Algorithm 3. CM-03 query-aware flow diffusion.

Cross-modal fusion strategies (RQ2)

Fusion decides how the three evidence streams are presented. To isolate it completely, every method receives the **byte-identical cached evidence bundle** of each question (the top-5 evidence of B1, B2 and B3 plus the variant's overlays) and may not retrieve. Fig. 10 contrasts the context layouts.

CF-B0 (raw concatenation) writes all items in source order until the budget is reached. **CF-B1 (source-separated)** writes labelled TEXT, GRAPH and STRUCTURED sections, each limited to 8,000 characters.

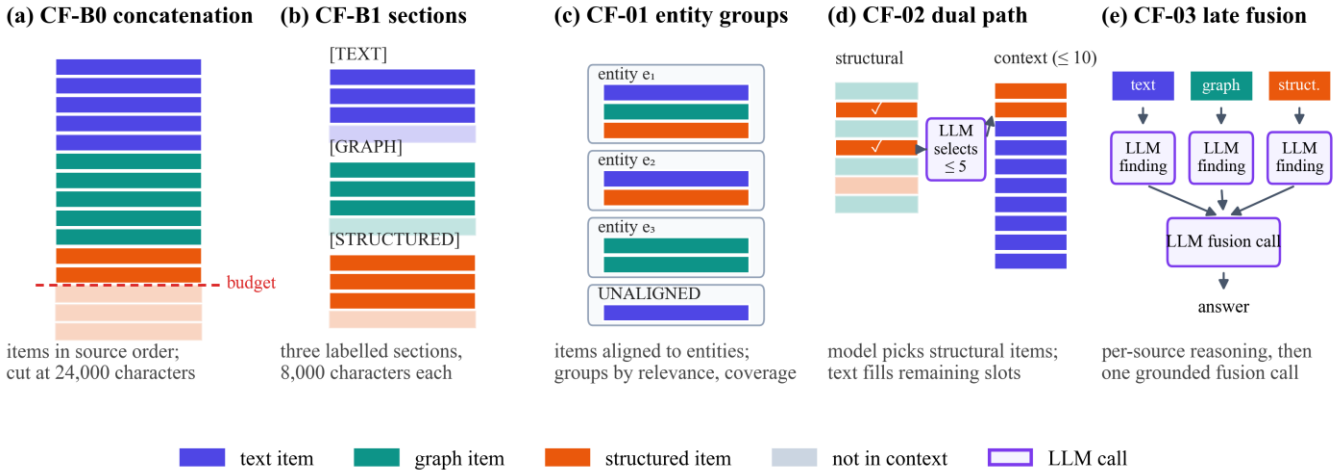


Fig. 10. Context layouts of the five fusion strategies of RQ2 on the same evidence bundle. (a) Raw concatenation in source order, cut at the budget. (b) Three labelled sections with a third of the budget each. (c) Evidence grouped by aligned entity. (d) An LLM selects up to five structural items; text fills the remaining slots. (e) One reasoning call per source, then a grounded fusion call.

CF-01 (KAG-inspired entity alignment)[6] attaches every evidence item to an entity in a fixed order (explicit identifier, then a unique exact normalised alias, then a registry lookup restricted to the item's entity type); unresolved items form UNALIGNED groups. Groups are ordered by their best

member relevance, then by the number of sources they cover, then by entity identifier, and written whole until the budget is full. No model invents entities.

CF-02 (SAG-inspired dual path)[7] treats graph and structured items as *structural* and text items as *semantic* candidates. One selector call reads a compact manifest of the structural candidates (one line each: identifier, source, relevance, the first 200 characters, provenance) and returns at most five identifiers; one schema repair is allowed, otherwise the five most relevant items are used. Text items then fill the context to ten items in retriever order.

CF-03 (MegaRAG-inspired late fusion)[8] reasons over each non-empty source separately and fuses the findings (Algorithm 4).

Input: question q, bundle with sources $\{T, G, S\}$
Output: answer $A(q)$
Step 1. **for each** source s with evidence **do**
- $f_s \leftarrow$ LLM(q, evidence of s only) as {candidate_answer, supporting_ids, confidence}
- **if** supporting_ids $\not\subset$ IDs(s) **or** the call fails **then** $f_s \leftarrow$ NO_EVIDENCE

Step 4. $A(q) \leftarrow$ LLM(q, $\{f_s\}$), citing only papers supported by some f_s
Step 5. **if** the fusion call fails **then** retry once, **else return** $A(q)$

Algorithm 4. CF-03 late fusion with grounded findings.

Fusion cannot recover evidence that was never retrieved. Before any model call, each bundle b with evidence papers $P(b)$ was therefore assigned a supportability class, used only for analysis:

$$\text{class}(b) = \begin{cases} \text{FULLY} & \text{if } G(q) \subseteq P(b) \\ \text{PARTIALLY} & \text{if } G(q) \cap P(b) \neq \emptyset \\ \text{UNSUPPORTABLE} & \text{otherwise} \end{cases} \tag{10}$$

Retrieval-control policies (RQ3)

Retrieval control decides which sources to query, in which order and when to stop. All policies share the same tools (the unchanged B1–B3), a cumulative evidence memory and a fixed downstream context (labelled sections of 8,000 characters for the sources actually queried), so the policy is the only variable. Fig. 11 shows their control flow.

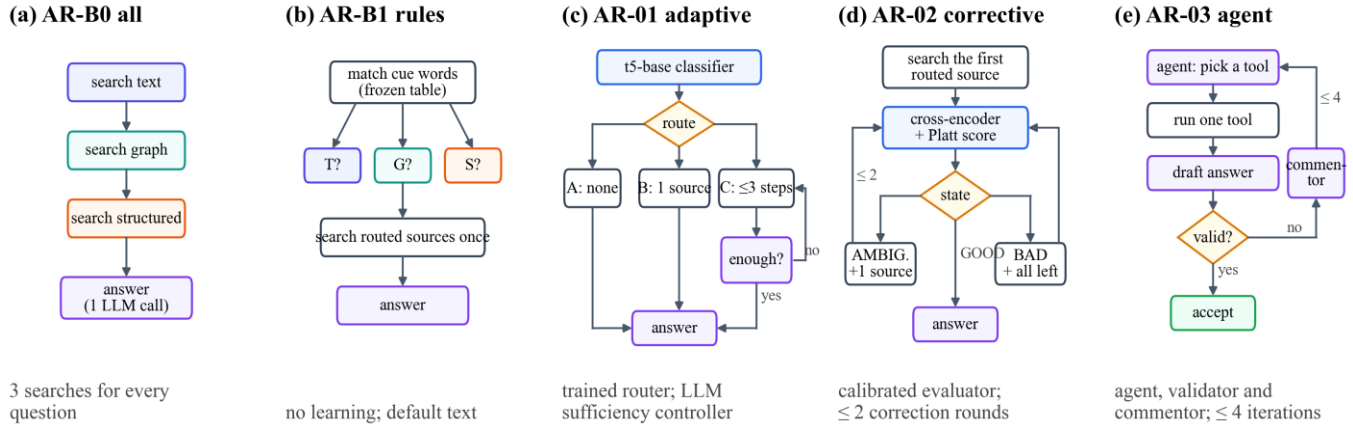


Fig. 11. Control flow of the five retrieval-control policies of RQ3. (a) Always query all three sources. (b) Frozen keyword cues select the sources. (c) A trained classifier chooses no retrieval, one source or multi-step retrieval with an LLM sufficiency check. (d) A calibrated cross-encoder rates the evidence and triggers corrections. (e) An agent chooses one tool per step; a validator accepts or a commentor returns feedback.

AR-B0 always queries text, graph and structured with the raw question. **AR-B1 (rule router)** normalises the question (NFKC, lowercase) and matches a frozen cue table: text cues (for example "abstract", "mentions"), graph cues ("author", "affiliated", "cites") and structured cues ("year", "rank", "at least", "≥"). It queries every matched source once in the order text, graph, structured; with no match it queries text. The cue table was frozen before any result and serves as the base router of AR-01 and AR-02.

AR-01 (Adaptive-RAG-inspired)[9]. A t5-base model[45] predicts a route: A (no retrieval), B (the router's first source) or C (up to three steps, with an LLM controller returning {sufficient,

next_subquery} after each step). Training labels come from a retrieval-only oracle on separate training questions:

$$\ell(q) = \min\{\ell \in (A, B, C) : \mathrm{succ}_\ell(q)\} \tag{11}$$

where A succeeds only for an empty gold set, B succeeds if the router's first source retrieves every gold paper, and C if up to three sources do; the label defaults to C. A hash-based partition of the 8,049 training questions gave 7,302 training and 747 development questions; the model (AdamW, learning rate 3×10^{-5}, weight decay 0.01, batch size 16, at most 10 epochs with early stopping) kept the epoch with the lowest development loss (epoch 7, loss 0.2170). Fig. 12 shows the training curve and the oracle labels; most training questions need multi-step retrieval.

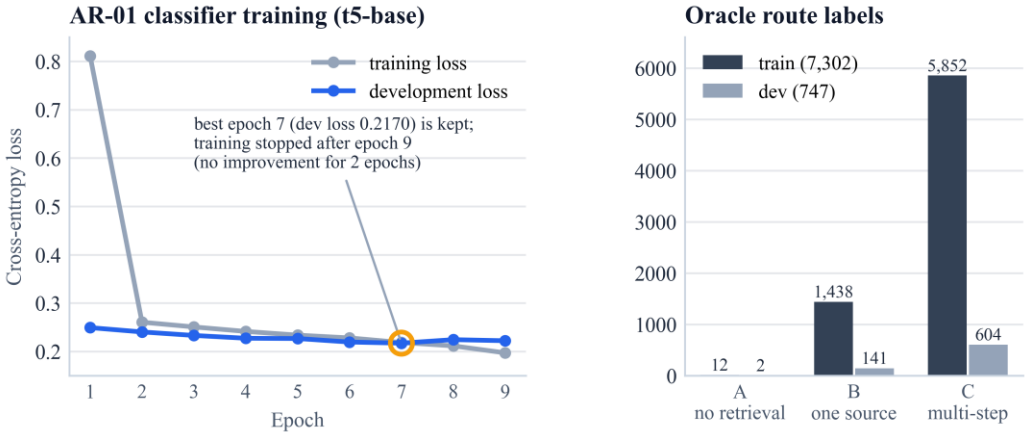


Fig. 12. AR-01 classifier: training and development loss per epoch (left; the kept epoch is circled) and the oracle route labels of the training and development partitions (right).

AR-02 (CRAG-inspired)[10]. A MiniLM cross-encoder trained on MS MARCO[46,47] scores each (question, evidence) pair; Platt scaling[48], fitted on the development partition only (7,498 labelled items from 745 questions), maps a raw score s to a probability

$$p = \frac{1}{1 + e^{-(a\,s+b)}}, \qquad a = 0.376, \qquad b = 0.102 \tag{12}$$

and the evidence state follows

$$\mathrm{state}(q) = \begin{cases} \mathrm{GOOD} & \text{if } \max_i p_i \geq U \\ \mathrm{BAD} & \text{if } \max_i p_i \leq L \\ \mathrm{AMBIGUOUS} & \text{otherwise} \end{cases} \tag{13}$$

with $L = 0.40$ and $U = 0.75$ chosen by macro-F1 from a frozen grid on 2,235 development states (Fig. 13, Algorithm 5).

Input: question q; router order; thresholds L, U
Output: answer $A(q)$
Step 1. query the router's first source; rounds ← 0
Step 2. **loop**
- p_i ← Platt(cross-encoder(q, item i)) for all items; state by (13)
- **if** state = GOOD **or** rounds = 2 **or** no source is left **then break**
- **if** AMBIGUOUS **then** add the next unqueried source **else** add all remaining
- rounds ← rounds + 1

Step 7. **return** $g(q$,fixed context)

Algorithm 5. AR-02 calibrated corrective retrieval.

AR-03 (AGENT-G-inspired)[11]. An LLM agent chooses one action per iteration (a source with a query, or stop); after each tool call a candidate answer is generated and an LLM validator, which never sees gold data, accepts or rejects it; on rejection an LLM commentor returns feedback (Algorithm 6).

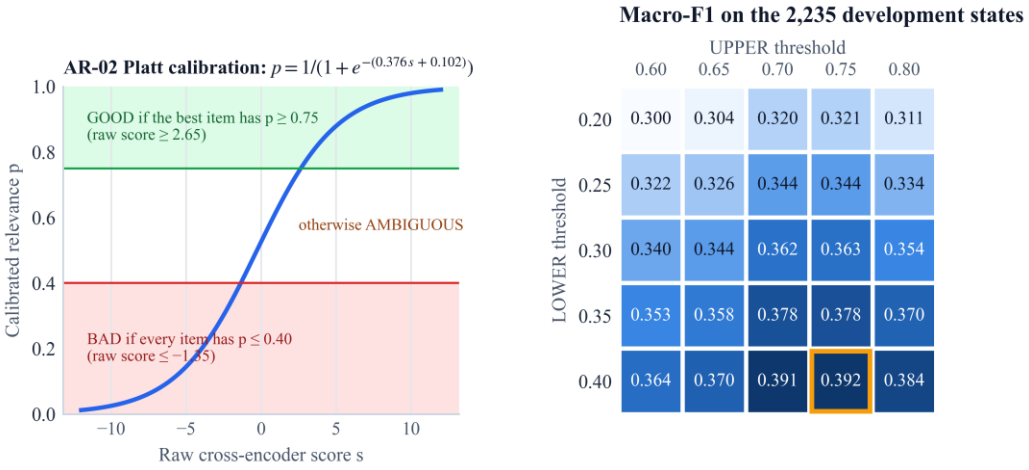


Fig. 13. AR-02 calibration: the fitted Platt curve with the GOOD and BAD regions (left) and the macro-F1 of every threshold pair on the development states, with the chosen pair outlined (right).

Input: question q; at most $T = 4$ iterations
Output: answer $A(q)$ or INSUFFICIENT_EVIDENCE
Step 1. **for** $i = 1, \ldots, T$ **do**
- action ← agent(q, feedback, memory) (one schema repair allowed)
- **if** action = stop **then return** $g(q$,memory)
- run exactly one tool; add new evidence to memory
- Â ← $g(q$,memory); verdict ← validator(q, Â, memory)
- **if** verdict accepts **then return** Â
- feedback ← commentor(q, action, verdict); stop on repeated empty actions

Step 8. **return** INSUFFICIENT_EVIDENCE (empty answer)

Algorithm 6. AR-03 agent–validator–commentor loop.

Integration (RQ4)

The integrated system **INT-1** chains the selected policy, context management on graph evidence and the selected fusion method, whose answer becomes the system's answer. The components were selected by rules fixed before any RQ2 or RQ3 result was analysed: context management was fixed to CM-B1 for cost, and the policy and fusion method are the Pareto-optimal strategies with the highest answer F1,

$$m^* = \arg\max_{m \in \mathcal{P}} F_1(m) \tag{14}$$

with ties broken by fewer tokens and then by a fixed order. INT-1 is compared with AR-B0 and with the selected policy alone on the same 28 questions. Algorithm 7 summarises the complete procedure, from the frozen data to the integrated system.

Input: pinned corpus $\mathcal{D}$; benchmark $\mathcal{Q}$; retrievers B1–B3; LLM g; budget B; strategy sets $\mathcal{K}$, Φ, Π
Output: per-question records, statistics, winners and the integrated system INT-1
Step 1. build V_T, V_G, V_S from $\mathcal{D}$ and audit isolation; construct and verify $\mathcal{Q}$ (Algorithm 1)
Step 2. **for each** study RQ1, RQ3, RQ2 (one variable at a time) **do**
- freeze the execution plan: hashes of code, configuration, prompts, inputs, artefacts and cohort
- **call phase:** for each question, run every strategy and store each distinct request once
- **replay phase:** rerun every strategy with a store-only client; stop on any differing request
- compute metrics per question; paired family-clustered tests with Holm correction (Algorithm 8)

Step 7. select the policy and fusion method by the frozen rule (14); fix context management to CM-B1
Step 8. run INT-1 = policy → CM-B1 → fusion, and its comparisons, with the same protocol
Step 9. rank all strategies on their Pareto axes; export the checksummed results bundle
Step 10. **return** records, tests, Pareto sets, winners, INT-1

Algorithm 7. The controlled multi-modal GraphRAG study.

Execution protocol and reproducibility

Language-model experiments are hard to reproduce: outputs can change between runs, runs can be interrupted, and a small change in a prompt or setting can silently change a result. The project therefore separates *producing* model responses from *computing* results and freezes everything that could matter (Fig. 14). Before a study runs, its execution plan records the SHA-256 of every code file the study imports, every configuration, prompt and input file, every model artefact and the exact list of questions and methods; a runner refuses to start or resume if any hash differs. Every model call goes through a client that identifies it by the SHA-256 of its full request payload; the first time, the request is sent and the response, token usage and timing are stored durably, and an identical payload is answered from the store byte for byte. Calls are made question by question for all methods (question-major order), after which the unchanged evaluation runner replays each method with a store-only client that stops on any request that differs from the stored one, a built-in byte-identity check of the whole pipeline. The runner writes one record per question and method, atomically. Failures are recorded, never silently retried, and a cooling guard pauses requests while the graphics processor is hot; such pauses are never counted as latency. At every checkpoint a preservation audit re-hashes the 695 frozen payload files (7.17 GB).

Evaluation metrics

For a question with returned set A and gold set G,

$$P = \frac{|A \cap G|}{|A|}, \qquad R = \frac{|A \cap G|}{|G|}, \qquad F_1 = \frac{2PR}{P+R} \tag{15}$$

with the conventions that an empty prediction for an empty gold set scores 1 and an empty prediction for a non-empty gold set scores 0; exact match is $\mathbb{1}[A = G]$. Evidence quality is measured by the mean reciprocal rank of the first correct paper in the context order,

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q} \tag{16}$$

and by gold coverage, the share of correct papers present in the model's context:

$$\mathrm{GC}(q) = \frac{\left|G(q) \cap \mathrm{IDs}\big(c(q)\big)\right|}{|G(q)|} \tag{17}$$

For retrieval control, with S the set of queried sources and R the sources a question requires (known from its construction and used only for evaluation),

$$\mathrm{SF}_1 = \frac{2|S \cap R|}{|S| + |R|}, \ \mathrm{WR} = \frac{|S \setminus R|}{|S|} \tag{18}$$

and stopping accuracy is 1 when the policy stops exactly at the first step after which all required sources have been queried. Cost is measured by input and output tokens over all model calls, model and retrieval calls, and intrinsic latency

$$L = t_{\mathrm{ret}} + t_{\mathrm{LLM}} + t_{\mathrm{loc}} \tag{19}$$

the sum of retrieval time, model compute time and local computation measured during replay; waiting is excluded. Table 6 summarises the metrics.

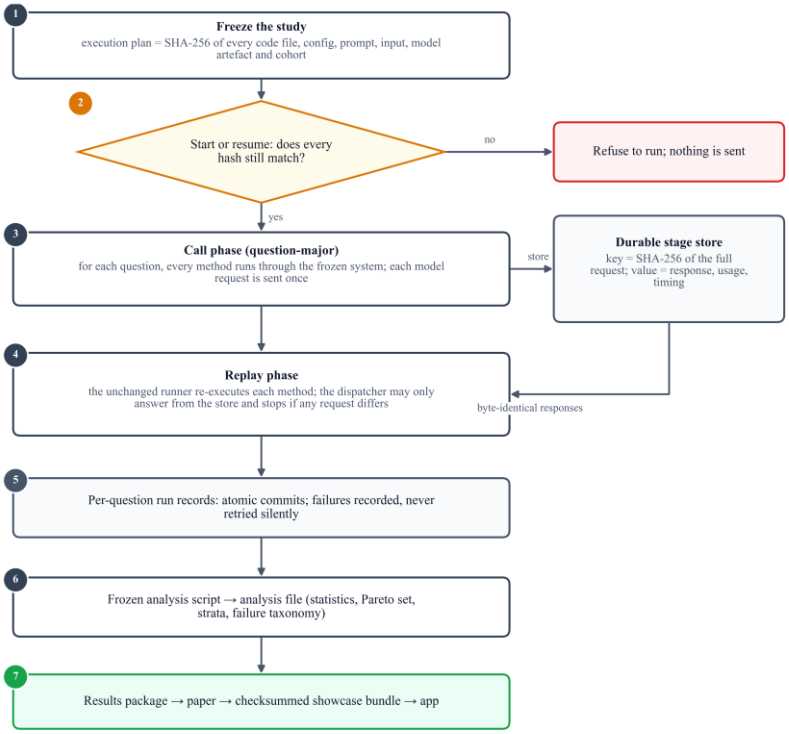


Fig. 14. The execution protocol: freeze, check, call once per request in question-major order, store every response, replay with the unchanged runner, then analyse with a frozen script.

Metric	Definition	
Answer F1	set F1 of returned vs. gold papers, (15)	↑
Exact match (EM)	returned set equals gold set	↑
MRR	reciprocal rank of the first correct paper, (16)	↑
Gold coverage (GC)	share of gold papers in the context, (17)	↑
Source F1 (SF1)	F1 of queried vs. required sources, (18)	↑
Wasted retrieval (WR)	share of queried sources not needed, (18)	↓
Stopping accuracy	stopped exactly when all needed sources were in hand	↑
Tokens	input + output tokens over all model calls	↓
Calls / searches	model calls / retriever calls per question	↓
Latency	retrieval + model compute + local time, (19)	↓

Table 6. Evaluation metrics (↑ higher is better, ↓ lower is better).

Statistical analysis

Two methods are always compared on the same questions, so the analysis looks at the difference per question rather than at two averages. Variants of one base question share their conditions and usually behave alike, so every test resamples or permutes whole families. For a pair of methods (A, B) and a metric, let $d(q)$ be the per-question difference, S_f its sum over the questions of family f and n_f their number. Confidence intervals come from a family bootstrap[13,14] with 10,000 resamples (seed 42):

$$\hat{\Delta}_b = \frac{\sum_{f \in \mathcal{B}_b} S_f}{\sum_{f \in \mathcal{B}_b} n_f} \tag{20}$$

where each resample $\mathcal{B}_b$ draws F families with replacement; the 2.5th and 97.5th percentiles of the 10,000 resampled means form the 95% interval. The two-sided p-value is an exact sign-flip test over all 2^F sign patterns of the F family sums[39],

$$p = \frac{1}{2^F} \sum_{\sigma \in \{\pm 1\}^F} \mathbb{1}[T(\sigma) \geq T(1)] \tag{21}$$

with the test statistic

$$T(\sigma) = \left| \sum_{f=1}^{F} \sigma_f S_f \right| \tag{22}$$

and the k pre-specified pairs of each metric are adjusted with Holm's method[12]:

$$\tilde{p}_{(i)} = \min \left\{ 1, \max_{j \leq i} (k - j + 1) p_{(j)} \right\} \tag{23}$$

A difference is called significant only if $\tilde{p} < 0.05$. Fig. 15 recomputes one real comparison from the recorded per-question results and reproduces the reported p-value and interval exactly.

A method is Pareto-optimal if no other method is at least as good on every frozen axis of its study and strictly better on one. The axes are answer F1, MRR, input tokens and latency for RQ1; answer F1, input tokens and latency for RQ2; and answer F1, source F1, tokens and latency for RQ3. To name one winner per study, each method is ranked from 1 to 5 on every axis (ties share the average rank) and the ranks are summed:

$$R(m) = \sum_{a \in \mathcal{A}} r_a(m), \qquad m^* = \arg\min_m R(m) \tag{24}$$

with ties broken by higher answer F1 and then fewer tokens. As a robustness check, the mean of min–max scaled axis scores must pick the same winner:

$$s(m) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \frac{v_a(m) - v_a^{\min}}{v_a^{\max} - v_a^{\min}} \tag{25}$$

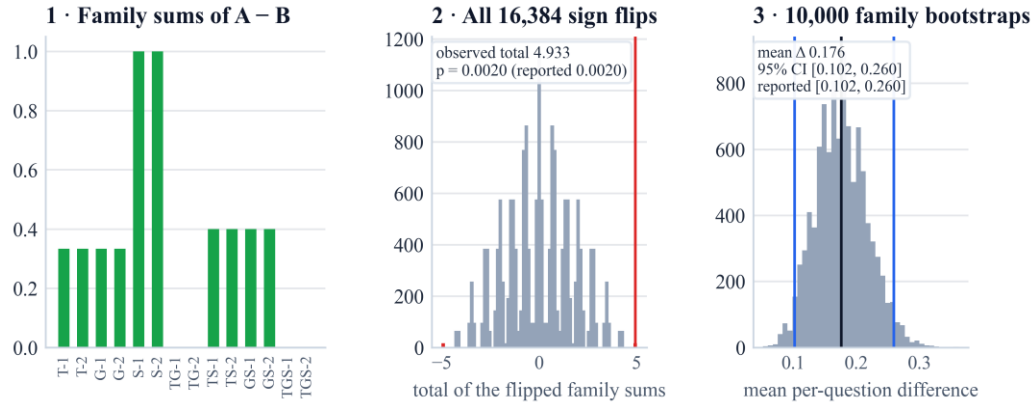


Fig. 15. The statistics of one real comparison (AR-B1 − AR-B0, source-selection F1, 14 families): (1) the family sums, (2) the totals of all 16,384 sign-flip patterns with the observed total in red, and (3) the 10,000 family-bootstrap means with the 95% interval.

The overall ranking was defined after the results were known; it summarises the frozen means and is not a significance test (Algorithm 8).

Input: per-question records; pre-specified pairs; Pareto axes $\mathcal{A}$
Output: tests, Pareto set, winner
Step 1. **for each** metric **and** pair (A, B) **do**
- compute $d(q)$, family sums S_f, sizes n_f
- bootstrap interval by (20); exact sign-flip p by (21) and (22)
- Holm-adjust the p-values of the metric by (23)

Step 5. compute the Pareto set on $\mathcal{A}$
Step 6. rank every method on every axis; $R(m)$ by (24); check by (25)
Step 7. **return** tests, Pareto set, m^*

Algorithm 8. Family-clustered comparison and overall ranking of one study.

Implementation and deployment infrastructure

The research code is a Python package beside the untouched original demonstration, organised by layer: contracts and hashing, data building, benchmark construction, evaluation, baselines, the canonical context-management and fusion methods, the retrieval-control policies with their durable model client, and the integrated system; 81 command-line scripts build, audit, run, report and freeze, and a suite of 327 automated tests passed at the final checkpoint. All studies ran on one workstation with the language model served locally. Table 7 lists the frozen scientific settings and Table 8 the data, preprocessing and use of every component.

Component	Setting
LLM	gpt-oss:20b via Ollama 0.40.5; temperature 0; per-question seed; low reasoning; JSON; 1,024-token cap per call; 16,384-token window
Embeddings	all-MiniLM-L6-v2 exported to ONNX; 384 dimensions; 256 tokens; mean pooling; L2 normalisation
Retrieval	top-5 per source; B2: 3 seeds, depth 2, caps 2,000 nodes / 20,000 edges / 128 per node; DuckDB read-only
Context	24,000 characters; 8,000 per section
CM-01	10 seeds; 1 hop; Kruskal maximum spanning trees; bge-reranker-large
CM-02	node prizes 3-2-1, edge prizes 5…1; $c_e = 0.5$; pcst_fast 1.0.10
CM-03	20 seeds; mass 10; sink 1; $\varepsilon = 0.05$; at most 10^6 pushes; $\tau = 0.01$
CF-02	at most 5 structural items; 10 items in total; one schema repair
AR-01	t5-base; AdamW, learning rate 3×10^{-5}; best epoch 7 of at most 10
AR-02	ms-marco-MiniLM-L-6-v2; Platt $a = 0.376$, $b = 0.102$; $L = 0.40$, $U = 0.75$; at most 2 rounds
AR-03	at most 4 iterations; agent, answer, validator and commentor calls
Statistics	10,000 family-bootstrap resamples (seed 42); exact sign-flip; Holm

Table 7. Frozen implementation settings of the shared components and of each strategy.

The **presentation layer** turns the results into an interactive application (Fig. 16). A builder exports the frozen analyses, run records, stored requests, benchmark records and paper titles into nine files with a SHA-256 manifest and a bundle checksum; the application verifies every checksum at start-up and withholds any content whose file does not match. All on-screen text lives in one content file whose numbers are placeholders resolved from the bundle, so text and results cannot drift apart. The application is built with Streamlit, Altair charts and self-contained graph drawings with precomputed deterministic layouts; it runs in a *showcase* profile from the bundle alone and in a *full* profile that can also run the frozen integrated-system code live against the local model, with live answers labelled as demonstrations outside the results.

Component	Data / inputs	Preprocessing	Use in the system
Text view	700,244 titles and abstracts	exact title + abstract; MiniLM embeddings; exact FAISS index	B1 dense retrieval; CM-01 and CM-03 similarities
Graph view	1,872,968 nodes; 19,919,698 edges	official relation types; tagged inverse rows; memory-mapped CSR	B2 traversal; candidate pools of RQ1
Structured view	700,244 paper rows	nullable attributes; explicit venue precedence; NULL kept	B3 typed read-only SQL
Benchmark	13,571 questions; 167 generated	gold before wording; strict necessity; stress overlays; audits	cohorts of the four studies
Training partition	7,302 training questions	retrieval-only oracle route labels	AR-01 classifier
Development partition	747 questions; 7,498 labelled items	cross-encoder scores; Platt fit; threshold grid	AR-02 calibration and thresholds
Stage store	every model request and response	keyed by SHA-256 of the full payload	call phase, replay phase, live demonstrations
Results bundle	frozen analyses, run records, stored requests	deterministic export; per-file SHA-256 manifest	interactive application

Table 8. Datasets, preprocessing and use of each component of the system.

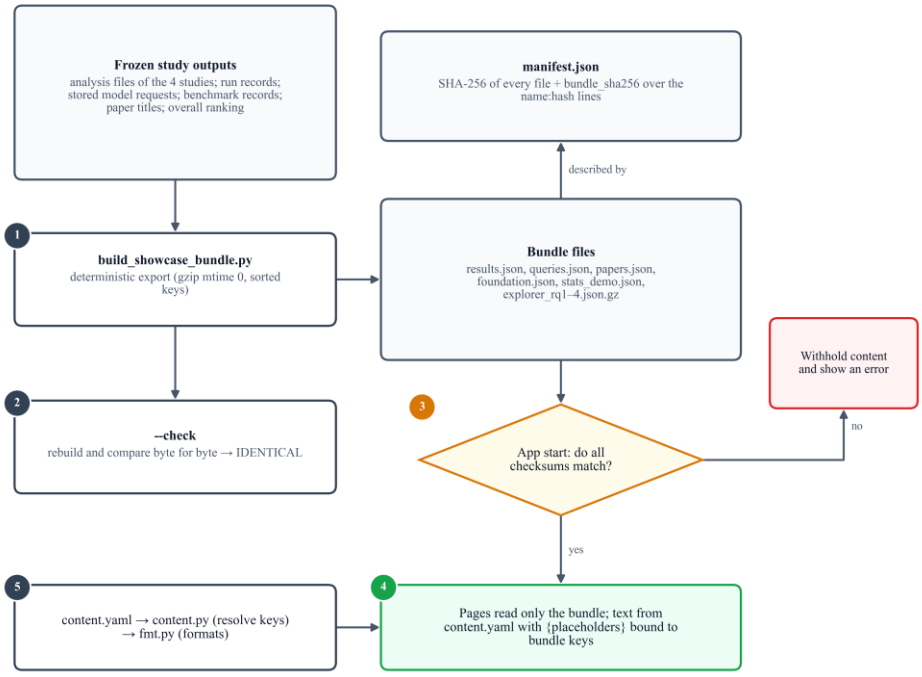


Fig. 16. The results bundle: the builder exports the frozen outputs into nine checksummed files; the application verifies every checksum at start-up, and the pages read only the bundle, with their text bound to it through the content file.

Experimental results

Experimental setup and cohorts

All studies use programmatically verified validation questions (Table 9). Cohort sizes were fixed before any outcome by the compute and time budget, and the four studies were run in the order RQ1, RQ3, RQ2, RQ4, producing 160, 140, 120 and 28 run records respectively (448 in total, 0 errors).

Study	Cohort	What changes	What stays fixed	Metrics
RQ1	32 questions: 8 families (2 each for G, TG, GS, TGS) × 4 variants (normal, noisy, conflict, unanswerable)	selection, order and pruning of graph evidence	graph candidate pool; text and structured evidence passed through; LLM; prompt; 24,000-character budget	F1 (primary); MRR, gold coverage, tokens, latency
RQ2	24 frozen bundles: 8 families (2 each for TG, TS, GS, TGS) × 3 variants (normal, noisy, conflict)	combination of text, graph and structured evidence	byte-identical bundles, no retrieval; LLM; prompt; 24,000-character budget	F1 (primary); EM, MRR, model calls, tokens, latency
RQ3	28 questions: 14 families (2 for each of the 7 source combinations) × 2 variants (normal, noisy)	which source, in which order, when to stop	retrievers B1–B3; 8,000 characters per queried source; LLM; prompt	F1 (primary); source-selection F1, wasted retrieval, stopping accuracy, calls, tokens, latency
RQ4	the 28 questions of RQ3	combination of the selected components	questions, data, retrievers, LLM, prompt	F1 (primary); source-selection F1, tokens, latency

Table 9. Design of the four studies.

Retrieval-only baseline evaluation

Before any study, a retrieval-only closeout ran the four baselines on the 56 generated validation questions (224 runs, no model calls, 0 errors; Table 10, Fig. 17). It established the ceiling that every later study inherits. B3 is exact on its own questions: on structured-only questions its recall, reciprocal rank and gold coverage are all 1.000. B2 is precise but capped: on graph-only questions every returned paper is correct, but the fixed top-5 holds only a small share of large answer sets. B1 finds almost nothing, because the text conditions are exact phrases while B1 searches by meaning;

this benchmark–baseline mismatch is reported as a limitation, and no exact-match shortcut was added after seeing it.

Baseline	Recall@5	Precision@5	MRR	nDCG@5	Gold coverage
B1 text	0.000	0.000	0.008	0.000	0.000
B2 graph	0.007	0.171	0.167	0.164	0.007
B3 structured	0.143	0.043	0.167	0.143	0.192
B4 all three	0.000	0.000	0.060	0.000	0.199

Table 10. Retrieval-only results of the four baselines (means over the 42 answerable generated validation questions).

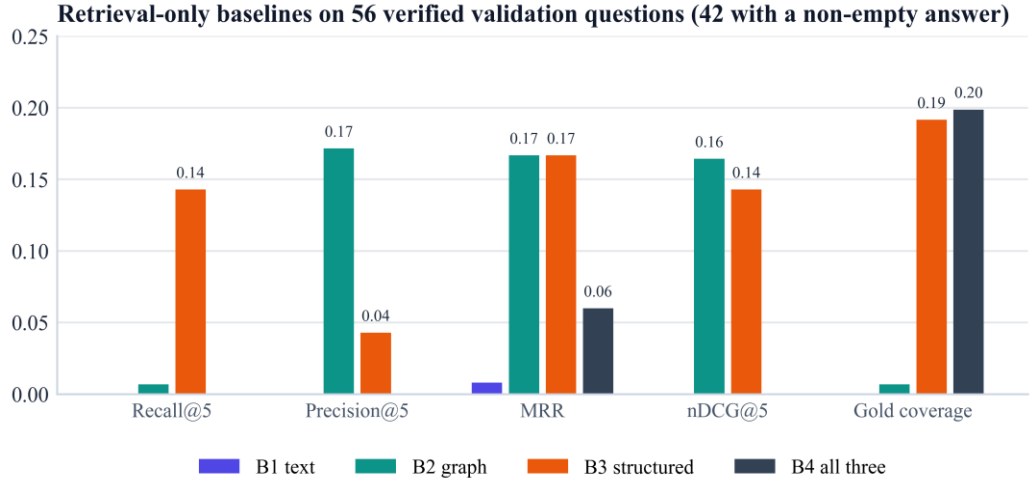


Fig. 17. Retrieval-only metrics of the four baselines on the 56 generated validation questions (means over the 42 with a non-empty answer). B1 bars are nearly absent because B1 found almost no correct papers.

Context management (RQ1)

Table 11 and Fig. 18 report the results on 32 questions. Answer quality was near its floor for every strategy: all five abstained correctly on the 8 unanswerable variants, which alone yields an F1 of 0.250, and CM-B0 and CM-03 returned no paper for any answerable question. CM-01 achieved the highest F1 (0.283; 0.045 on answerable questions) and MRR (0.359) at 3.5× the input tokens and 2.3× the latency of CM-B1. CM-B1 used about a third of the raw control's input tokens (1,416 vs. 4,499) with equal or better F1 and MRR. CM-02 was the fastest (6.5 s) but ranked correct papers lowest (MRR 0.187), and CM-03 kept the most correct papers in the context (gold coverage 0.338) yet was the slowest (40.7 s); its diffusion converged on only 5 of 32 questions.

Method	F1 ↑	F1 (ans.) ↑	EM ↑	MRR ↑	GC ↑	In tok. ↓	Lat. (s) ↓	Calls ↓
CM-B0	0.250	0.000	0.250	0.295	0.225	4,499	9.9	**1.00**
CM-B1 ★	0.264	0.019	0.250	0.353	0.221	**1,416**	7.6	**1.00**
CM-01 ★	**0.283**	**0.045**	0.250	**0.359**	0.175	4,982	17.8	**1.00**
CM-02 ★	0.251	0.001	0.250	0.187	0.085	1,464	**6.5**	**1.00**
CM-03	0.250	0.000	0.250	0.292	**0.338**	3,658	40.7	2.00

Table 11. RQ1 context management: means over 32 questions (★ Pareto-optimal; best value per column in bold).

No pair differed significantly in answer F1 (every Holm-adjusted p-value was 1.000): all differences came from the two graph-only families. The Pareto set is {CM-B1, CM-01, CM-02} (Fig. 19); CM-B1 dominates CM-B0 and CM-03. By source combination (Fig. 20), correct answers came only from the graph-only families, where CM-01 reached F1 0.384 against 0.307 for CM-B1; on the 18 answerable multi-source questions no strategy returned a correct paper.

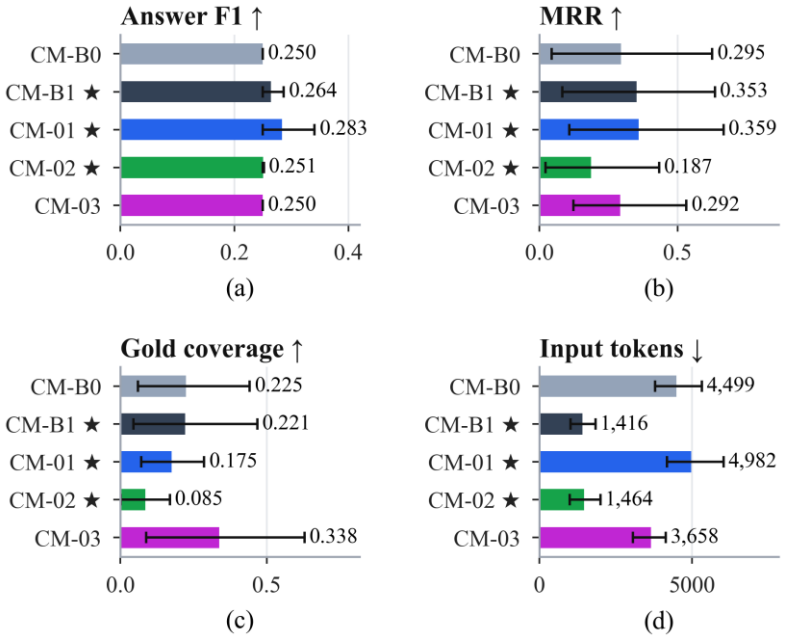


Fig. 18. RQ1 per strategy with family-clustered 95% bootstrap intervals: (a) answer F1, (b) MRR, (c) gold coverage, (d) input tokens. ★ marks the Pareto set.

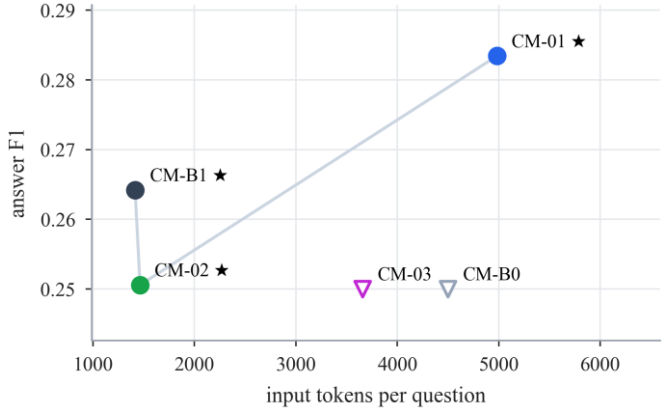


Fig. 19. RQ1 quality against cost. Circles are Pareto-optimal (★), triangles are dominated; up and to the left is better.

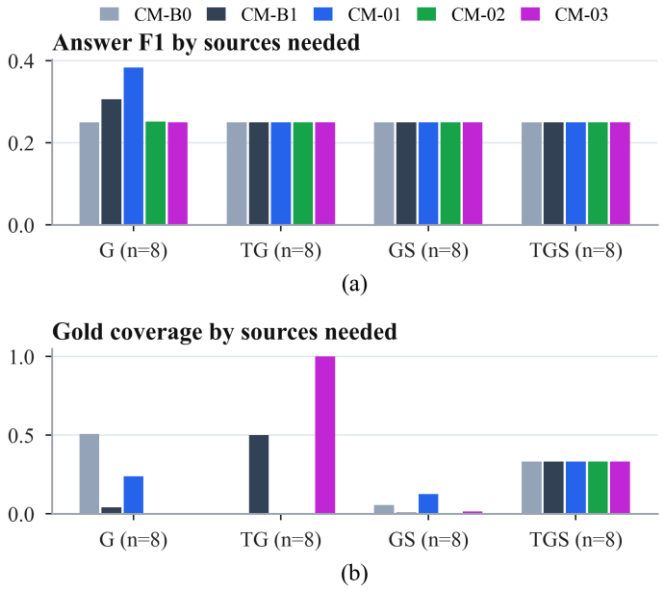


Fig. 20. RQ1 by the sources a question needs: (a) answer F1 and (b) gold coverage (8 questions per group). The value 0.25 in (a) is the share of correctly abstained unanswerable variants.

Cross-modal fusion (RQ2)

All five fusion strategies returned an empty answer on all 24 bundles, so answer F1 and exact match are 0.000 for every method (Table 12). Supportability, fixed before any call, explains most of this (Fig. 21a): 16 bundles contain no correct paper, 6 contain some and only 2 contain all; both fully supportable bundles are conflict variants in which the model judged that no paper met every condition. The strategies therefore differ only in cost and ranking. CF-B1 was the cheapest (3,485 input tokens) and fastest (8.4 s) and is the only Pareto-optimal method; CF-02 needed two calls and CF-03 3.75 on average, while CF-03 ranked a correct paper earliest (MRR 0.063). CF-01 resolved

every evidence item by its explicit identifier and never needed its registry lookup, and the CF-02 selector kept on average only 0.58 structural items per bundle.

Method	F1 ↑	EM ↑	MRR ↑	In tok. ↓	Out tok. ↓	Calls ↓	Lat. (s) ↓
CF-B0	0.000	0.000	0.029	4,097	**74**	**1.00**	11.0
CF-B1 ★	0.000	0.000	0.029	**3,485**	76	**1.00**	**8.4**
CF-01	0.000	0.000	0.029	4,644	84	**1.00**	9.1
CF-02	0.000	0.000	0.000	3,542	162	2.00	12.6
CF-03	0.000	0.000	**0.063**	4,455	288	3.75	19.6

Table 12. RQ2 cross-modal fusion: means over 24 bundles (★ Pareto-optimal; best in bold).

No RQ2 difference reached significance. Several cost differences had the same sign in all eight families; for example, CF-03 used 2.75 more calls per bundle than CF-B1 (unadjusted $p = 0.008$, the minimum attainable with eight families), but with seven pairs per metric their Holm-adjusted p-values were at least 0.055 (see the statistical power analysis in the Discussion).

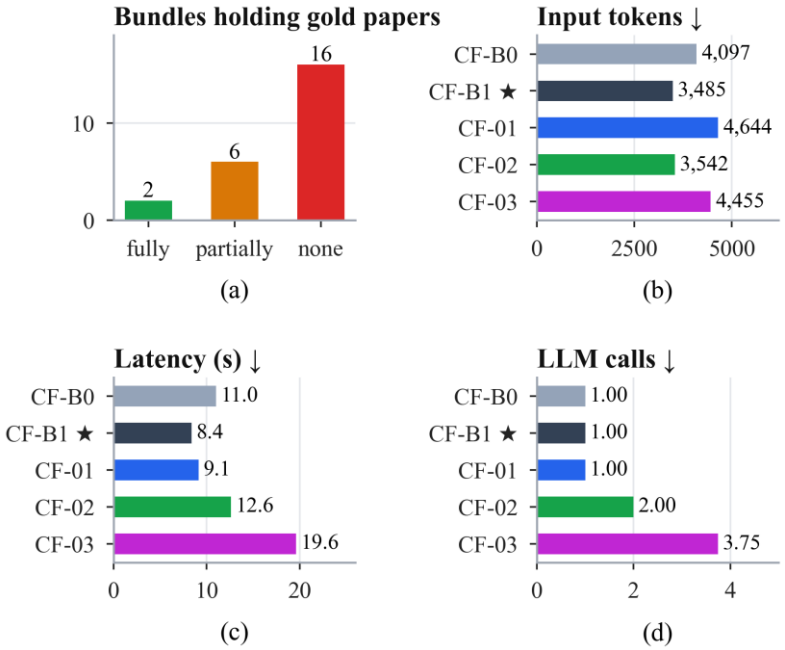


Fig. 21. RQ2: (a) supportability of the 24 bundles, fixed before any model call; (b) input tokens, (c) latency and (d) model calls per bundle.

Retrieval control (RQ3)

All five policies produced identical answers (F1 0.154, EM 0.143), so efficiency separated them (Table 13, Fig. 23). The rule router selected sources best (source F1 0.876) at the lowest cost (1,622 tokens, 8.7 s) and is the only Pareto-optimal policy. Querying every source wasted 43% of the searches. The agent wasted almost none (0.012) but needed 6.54 model calls per question and was the only policy to miss required sources (0.179).

Method	F1 ↑	SF1 ↑	WR ↓	Stop ↑	Miss ↓	Search ↓	Calls ↓	Tokens ↓	Lat. (s) ↓
AR-B0	0.154	0.700	0.429	**0.571**	**0.000**	3.00	**1.00**	2,251	10.6
AR-B1 ★	0.154	**0.876**	0.190	**0.571**	**0.000**	2.14	**1.00**	**1,622**	**8.7**
AR-01	0.154	0.843	0.238	**0.571**	**0.000**	2.43	2.71	4,419	24.1
AR-02	0.154	0.807	0.286	**0.571**	**0.000**	2.57	**1.00**	1,906	10.6
AR-03	0.154	0.871	**0.012**	0.500	0.179	**1.86**	6.54	7,719	45.2

Table 13. RQ3 retrieval control: means over 28 questions (★ Pareto-optimal; best in bold).

With 14 families, RQ3 can detect differences (Fig. 22). The router chose sources significantly better than querying everything (+0.176 source F1, Holm-adjusted $p = 0.014$), with significantly less wasted

retrieval, fewer searches, fewer tokens (−629 per question) and lower latency. Against the router, the learned and agentic policies cost significantly more for the same answers: AR-01 used 2,796 and AR-03 6,096 more tokens per question (both Holm-adjusted $p = 0.001$), and even AR-02 used 283 more ($p = 0.031$); all three were also significantly slower. No policy selected sources significantly better than the router.

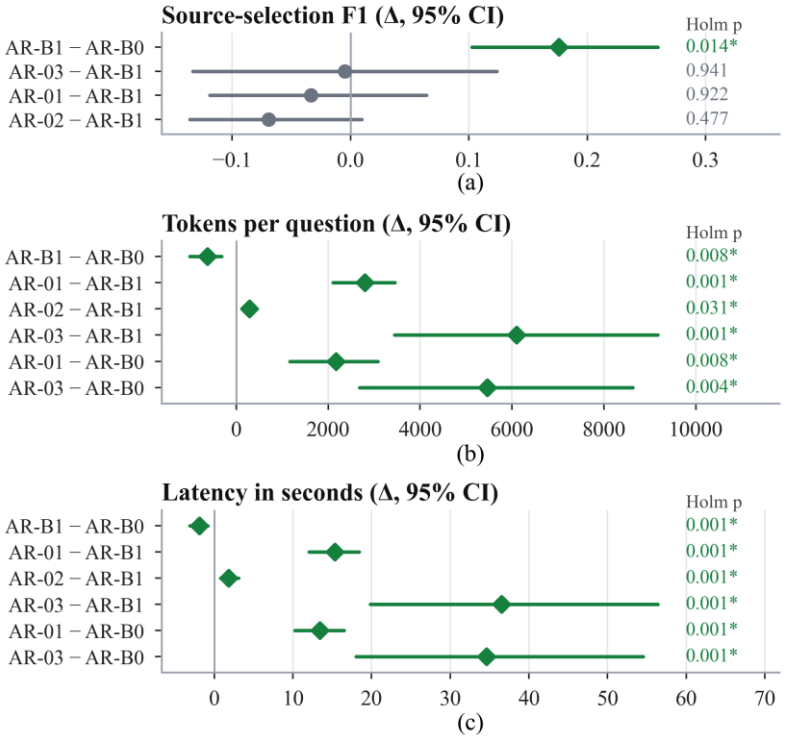


Fig. 22. RQ3 paired differences with 95% family-bootstrap intervals and Holm-adjusted *p*-values (* significant): (a) source-selection F1, (b) tokens, (c) latency.

Fig. 24 shows how the policies ended and Fig. 25 why they failed. The classifier of AR-01 chose the multi-step route for every question; its controller judged the evidence sufficient after one search on 8 questions, and the three-search limit ended the other 20. AR-02 rated the evidence GOOD on 6 questions, reached its round limit on 4 and exhausted the sources on 18. The validator of AR-03 accepted a draft answer on 22 questions, 18 of them wrongly, and the iteration limit ended the other 6. A retriever failure despite correct routing was recorded on 24 of the 28 questions for each of AR-B0, AR-B1, AR-01 and AR-02, their joint most frequent failure label; for AR-03 the most frequent label was a false acceptance by the validator. Fig. 26 shows the step-by-step traces of the router and the agent on the same question in the interactive application.

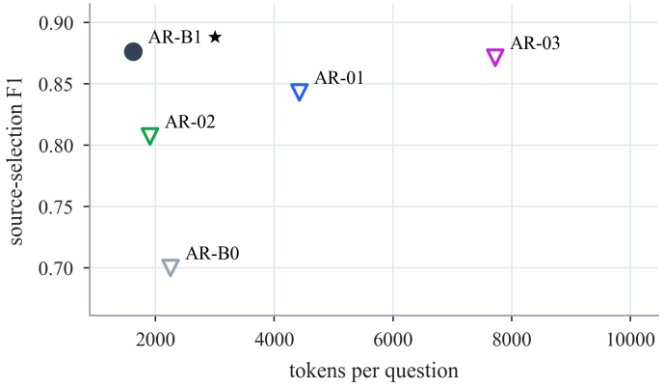


Fig. 23. RQ3 source selection against cost (answer F1 is identical for all policies). AR-B1 is the only Pareto-optimal policy.

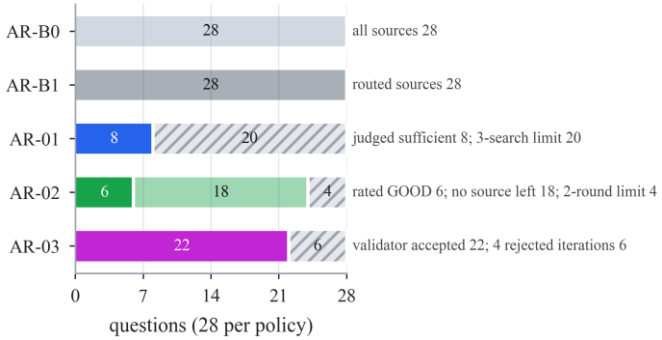


Fig. 24. RQ3 stop reasons over the 28 questions: stops decided by the policy (solid), fixed plans completed (light) and limits reached (hatched).

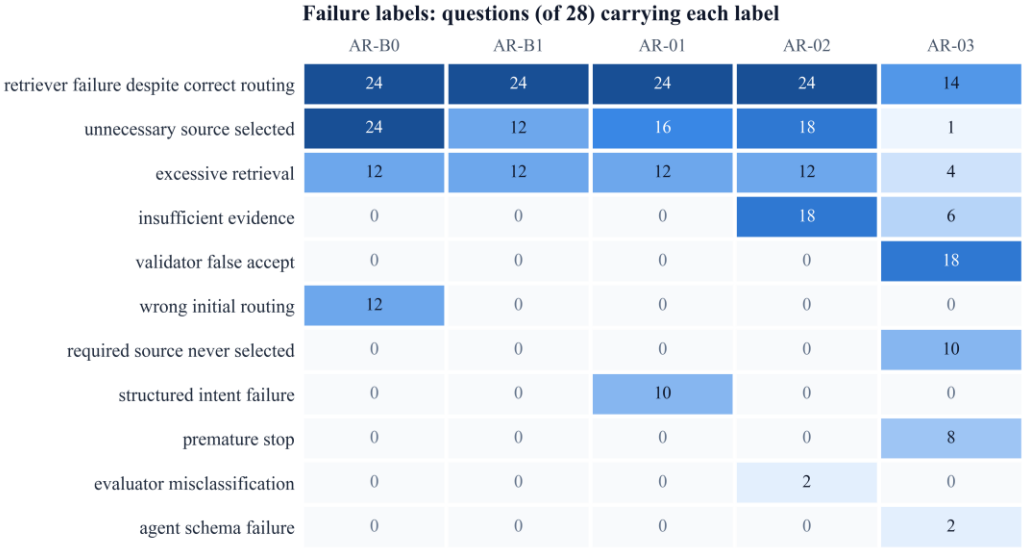


Fig. 25. RQ3 failure labels: number of questions (of 28) carrying each label for each policy.

Integrated system (RQ4)

The frozen rules selected **AR-B1 → CM-B1 → CF-B1**. INT-1 gave the same answers as both comparisons (F1 0.154), kept the router's source selection, significantly better than querying everything (+0.176 source F1, Holm-adjusted $p = 0.004$), and used 772 more tokens per question than the router alone ($p < 0.001$), because the CF-B1 fusion stage writes every evidence item as a full record with identifier, payload and provenance instead of the router's compact lines (7,964 vs. 5,287 context characters on average; Table 14, Fig. 27). Latency did not differ significantly. Under the frozen top-5 graph retriever, CM-B1 removed no graph evidence for this single-graph-call policy.

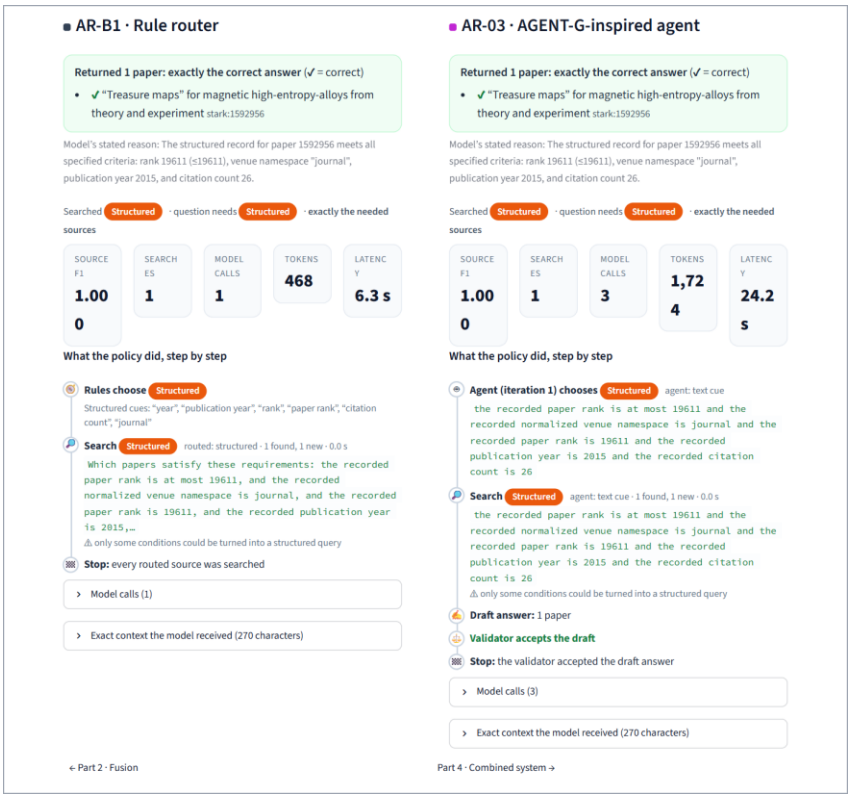


Fig. 26. Interactive application, retrieval control: the rule router (left) and the AGENT-G-inspired agent (right) on the same structured question, each with its answer, cost tiles and step-by-step trace. Both answer correctly; the agent needs three model calls and 24.2 s where the router needs one call and 7.3 s.

Method	F1 ↑	SF1 ↑	Search ↓	Tokens ↓	Lat. (s) ↓
AR-B0	0.154	0.700	3.00	2,251	10.6
AR-B1	0.154	**0.876**	**2.14**	**1,622**	**8.7**
INT-1	0.154	**0.876**	**2.14**	2,394	9.9

Table 14. RQ4 integrated system compared with AR-B0 and AR-B1 on 28 questions.

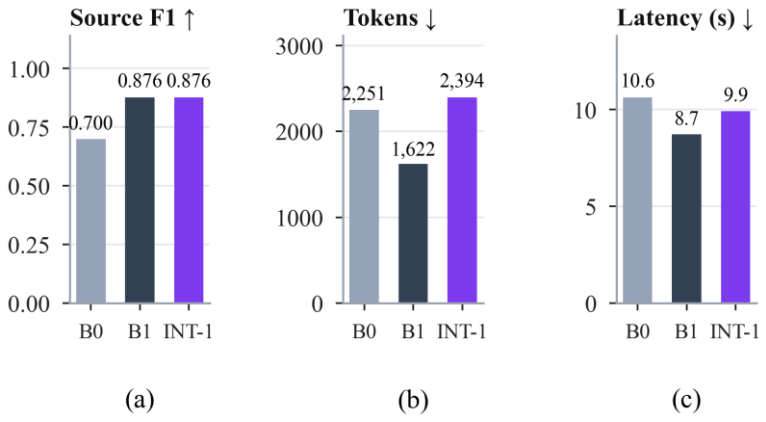


Fig. 27. RQ4: (a) source-selection F1, (b) tokens and (c) latency of AR-B0, AR-B1 and the integrated system INT-1.

Statistical summary and overall ranking

Table 15 lists every difference that survived Holm correction. All of them come from RQ3 and RQ4, whose 14 families give the tests enough resolution, and none concerns answer quality: every significant difference concerns source selection or cost.

Pair (A – B)	Metric	Δ	95% CI	*p* (Holm)
AR-B1 – AR-B0	SF1	+0.176	[0.102, 0.260]	0.014
AR-B1 – AR-B0	WR	−0.238	[−0.357, −0.131]	0.012
AR-03 – AR-B0	WR	−0.417	[−0.548, −0.286]	0.003
AR-B1 – AR-B0	searches	−0.86	[−1.21, −0.57]	0.014
AR-03 – AR-B0	searches	−1.14	[−1.64, −0.57]	0.031
AR-B1 – AR-B0	tokens	−629	[−1,010, −316]	0.008
AR-01 – AR-B0	tokens	+2,168	[1,162, 3,085]	0.008
AR-03 – AR-B0	tokens	+5,467	[2,684, 8,627]	0.004
AR-01 – AR-B1	tokens	+2,796	[2,101, 3,454]	0.001
AR-02 – AR-B1	tokens	+283	[130, 438]	0.031
AR-03 – AR-B1	tokens	+6,096	[3,447, 9,172]	0.001
AR-B1 – AR-B0	latency (s)	−1.9	[−3.1, −0.8]	0.001
AR-01 – AR-B0	latency (s)	+13.4	[10.2, 16.5]	0.001
AR-03 – AR-B0	latency (s)	+34.6	[18.0, 54.5]	0.001
AR-01 – AR-B1	latency (s)	+15.3	[12.1, 18.4]	0.001
AR-02 – AR-B1	latency (s)	+1.8	[0.8, 3.1]	0.001
AR-03 – AR-B1	latency (s)	+36.5	[19.8, 56.4]	0.001
INT-1 – AR-B0	SF1	+0.176	[0.102, 0.260]	0.004
INT-1 – AR-B1	tokens	+772	[596, 946]	< 0.001

Table 15. All paired differences that remain significant after Holm correction (RQ3 and RQ4; Δ = A − B with family-bootstrap 95% intervals; 14 families).

Table 16 and Fig. 28 report the overall ranking. CM-B1 wins RQ1 with a rank total of 7 (second on F1, MRR and latency, first on tokens), CF-B1 wins RQ2 (5; first on both cost axes with F1 tied) and AR-B1 wins RQ3 (6; first on source F1, tokens and latency). The scaled-score check selects the same winner in every study. Chained in pipeline order, the winners are exactly INT-1, which the frozen rule had selected before the ranking was defined. A quality-first rule would choose CM-01 in RQ1 and the same winners elsewhere.

Study	Winner	Runner-up	Scaled check	Quality-first
RQ1	CM-B1 (7)	CM-01, CM-02 (11)	CM-B1 (0.838)	CM-01
RQ2	CF-B1 (5)	CF-02, CF-B0 (9)	CF-B1 (1.000)	CF-B1
RQ3	AR-B1 (6)	AR-02 (11)	AR-B1 (1.000)	AR-B1

Table 16. Overall winners (rank totals over the Pareto axes; lower is better).

RQ1 context management

	F1	MRR	input tok.	latency	total	scaled
CM-B1	2	2	1	2	**7**	**0.838**
CM-01	1	1	5	4	11	0.667
CM-02	3	5	2	1	11	0.501
CM-B0	4.5	3	4	3	14.5	0.417
CM-03	4.5	4	3	5	16.5	0.246

RQ2 fusion

	F1	input tok.	latency	total	scaled
CF-B1	3	1	1	**5**	**1.000**
CF-02	3	2	4	9	0.858
CF-B0	3	3	3	9	0.746
CF-01	3	5	2	10	0.645
CF-03	3	4	5	12	0.388

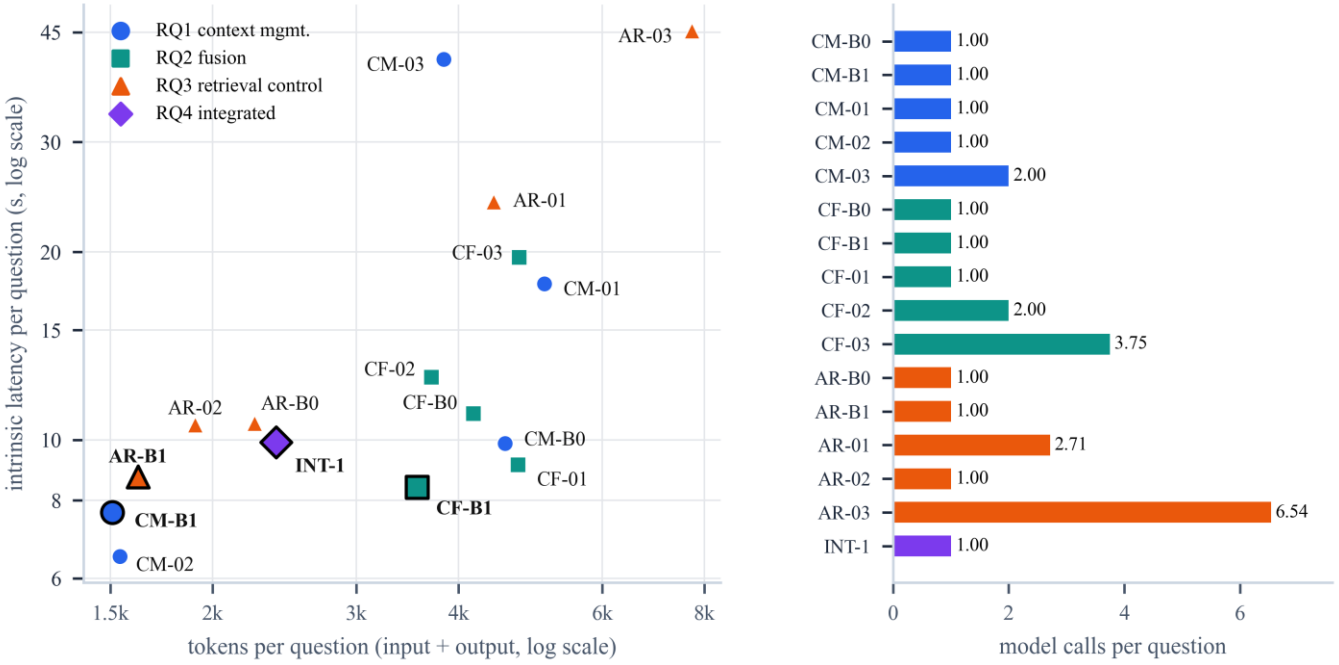
RQ3 retrieval control

	F1	source F1	tokens	latency	total	scaled
AR-B1	3	1	1	1	**6**	**1.000**
AR-02	3	4	2	2	11	0.878
AR-B0	3	5	3	3	14	0.711
AR-01	3	3	4	4	14	0.733
AR-03	3	2	5	5	15	0.493

Fig. 28. Overall ranking per study: rank on each Pareto axis (1 = best; darker is better), rank total and scaled score. The outlined row is the winner.

System-wide cost characteristics

Fig. 29 places all sixteen strategies on one cost map. Within each study the overall winner sits at or near the cheapest corner: CM-B1 and CM-02 need about 1,500 tokens and under 8 s per question, AR-B1 about 1,600 tokens and 8.7 s, and the integrated system adds the fusion stage's longer context but no extra model call. Cost grows with the number of model calls (Fig. 29b): the per-source reasoning of CF-03 (3.75 calls), the multi-step controller of AR-01 (2.71) and the agent loop of AR-03 (6.54) multiply tokens and latency, while the diffusion of CM-03 is slow because of its local computation (25.2 s per question for the context step alone) rather than its two calls.



(a) cost of every strategy (outlined, bold: overall winners) (b) model calls

Fig. 29. Cost of all sixteen strategies: (a) tokens and intrinsic latency per question on logarithmic axes, with the overall winners outlined; (b) model calls per question.

Interactive replay application

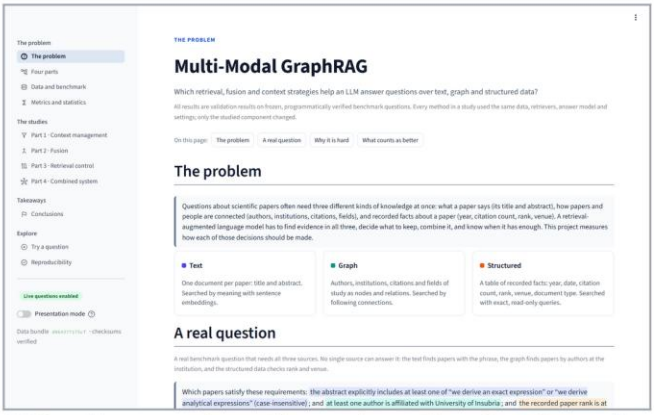
Every result in this paper can be inspected in the interactive application (Fig. 30). Its eleven pages follow the logic of the research: the problem and a real benchmark question; the four studies; the data and the benchmark; the metrics and statistics; one page per study with its strategies, results, statistical tests, findings and an explorer of the recorded runs; the conclusions; a page to replay any recorded pipeline on any question; and a reproducibility page (Table 17).

Page	Content
The problem	a real three-source question, why it is hard, what counts as better
Four parts	the pipeline and the one-variable-at-a-time design
Data and benchmark	the three views of one corpus, one paper seen through each view, the variants of a question
Metrics and statistics	metric definitions with an interactive calculator; the sign-flip test on real results; Pareto sets; protocol
Part 1 · Context management	strategies, results, tests, findings, breakdown, explorer with graph drawings
Part 2 · Cross-modal fusion	the same, plus the supportability of every bundle
Part 3 · Retrieval control	the same, plus stop reasons and step-by-step traces
Part 4 · Combined system	the three winners combined, results, explorer, final scoreboard
Conclusions	winners, the combined system, situations favouring other strategies, limitations
Try a question	replay of six recorded pipelines side by side; live runs in the full profile
Reproducibility	bundle checksum, verified files, frozen sources, run records, rebuild commands

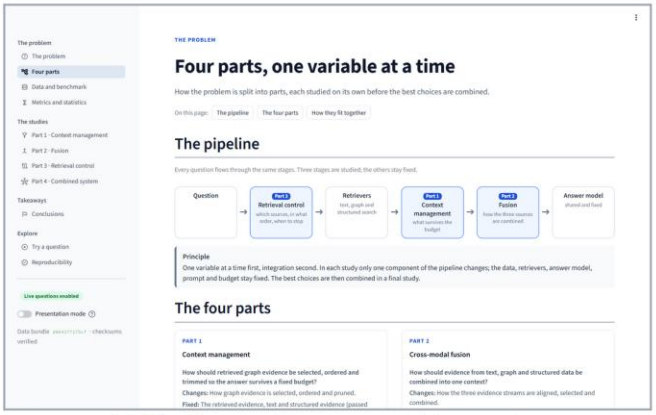
Table 17. Pages of the interactive application and what each shows.

The data page shows one paper through the three views, and the statistics page recomputes the sign-flip test from the recorded per-question differences (Fig. 31). Each study page ends with an explorer that shows, for any question and strategy, the answer with every returned paper marked correct or wrong, the cost tiles, the exact context the model received and, for graph evidence, an interactive drawing in which the correct answers, the returned papers and the retriever seeds are marked (Fig. 32). An empty answer is never shown as a bare failure: the application states whether

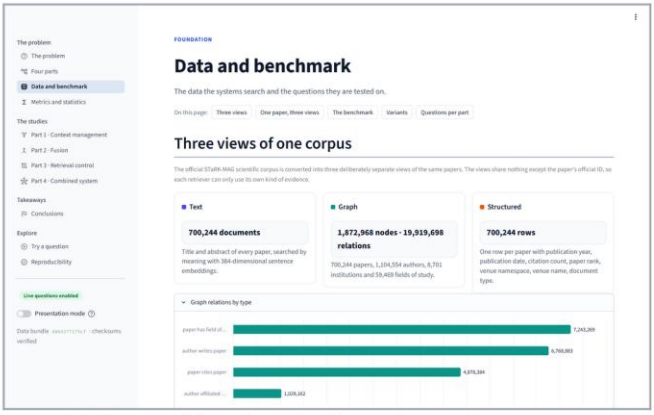
the retrievers never returned a correct paper, the strategy left it out of the context, or the model saw it and judged that no paper met every condition. Finally, any recorded pipeline can be replayed and compared side by side on any question, and the reproducibility page verifies every file of the bundle against its checksum (Fig. 33).



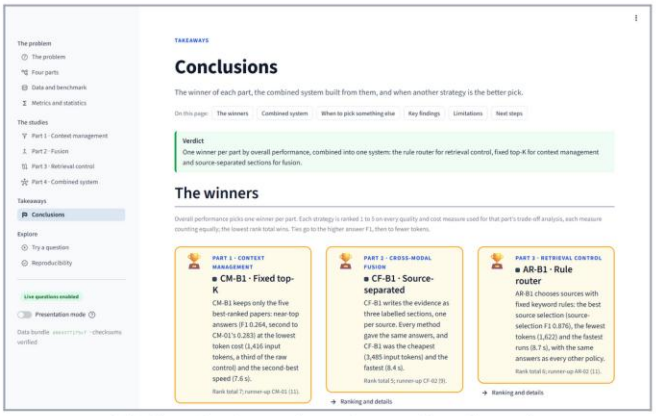
(a) Landing page: the problem and a real benchmark question



(b) The four studies, one variable at a time

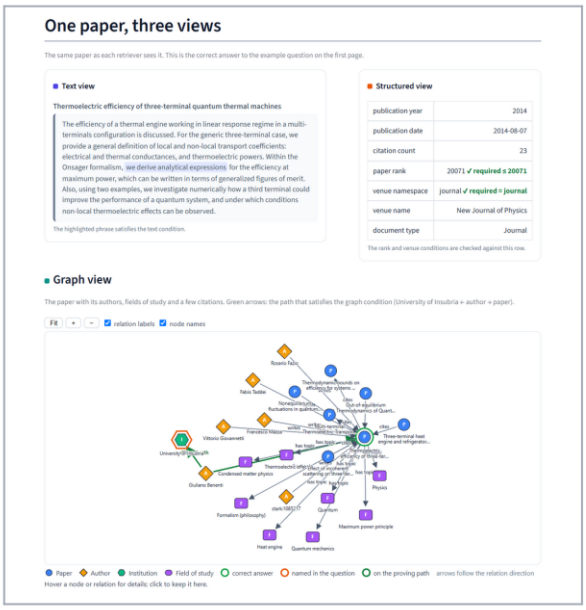


(c) Data and benchmark: three views of one corpus



(d) Conclusions: the winner of each study

Fig. 30. Pages of the interactive application, read only from the checksummed results bundle.

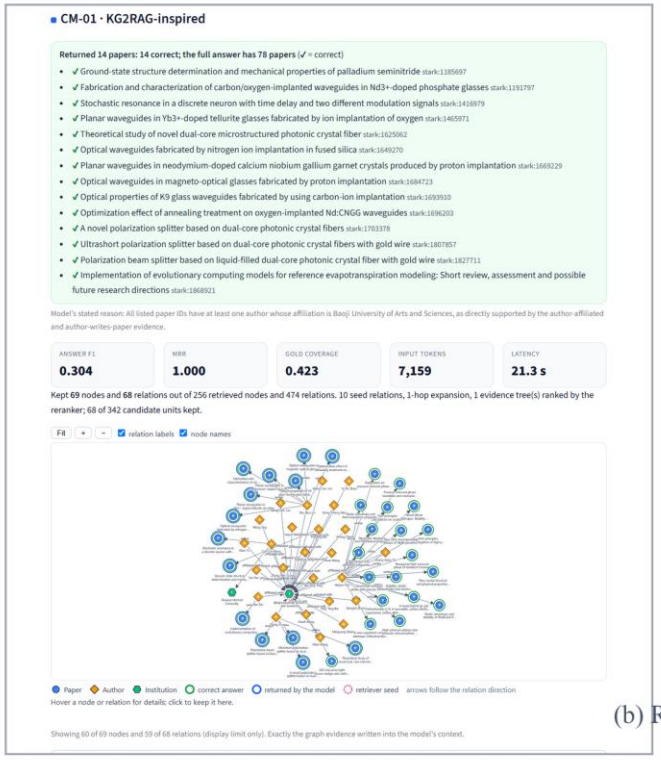


(a) One paper seen through the text, structured and graph views

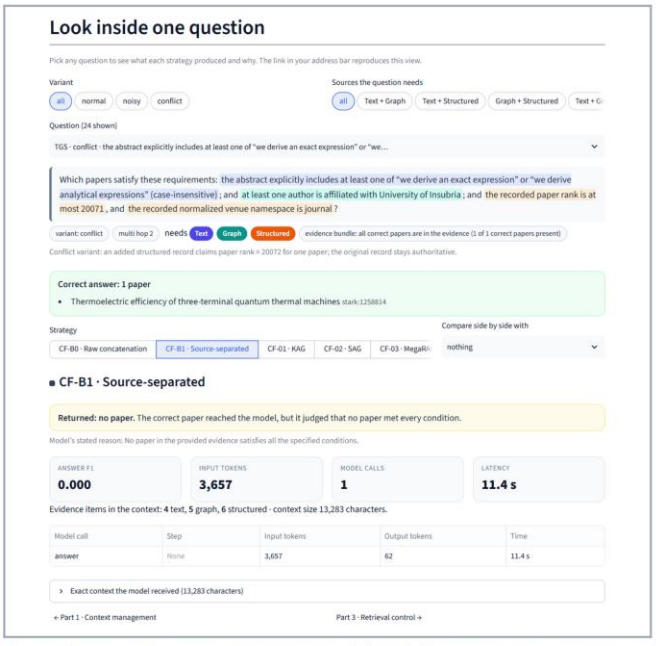


(b) The sign-flip test recomputed on real per-question results

Fig. 31. Interactive application: (a) one paper seen through the text, structured and graph views; (b) the sign-flip test recomputed from the recorded per-question results of one comparison.

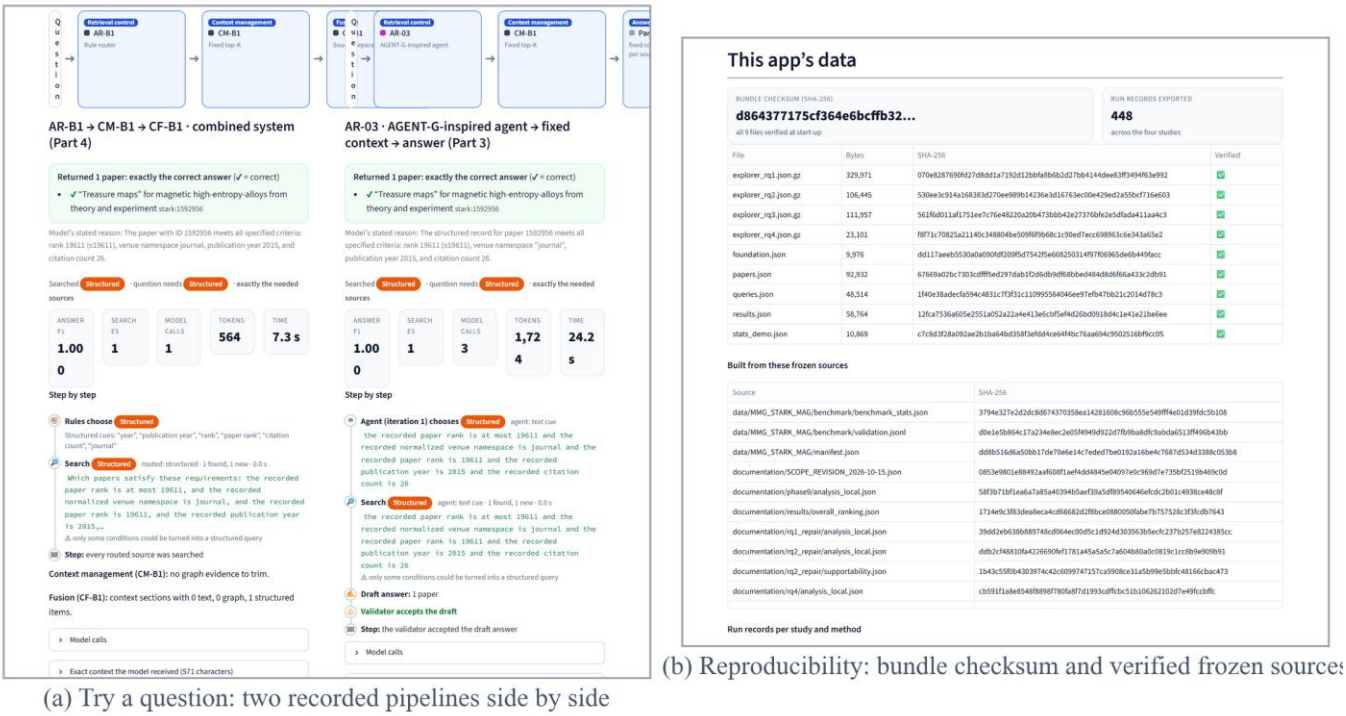


(a) RQ1 explorer: CM-01 context with the graph evidence drawn



(b) RQ2 explorer: an empty answer explained (correct paper seen, reje

Fig. 32. Interactive application, study explorers: (a) the CM-01 context of a graph-only question with the graph evidence drawn (14 papers returned, all correct); (b) an empty RQ2 answer explained: the correct paper reached the model, which judged that no paper met every condition.



(a) Try a question: two recorded pipelines side by side

(b) Reproducibility: bundle checksum and verified frozen sources

Fig. 33. Interactive application: (a) two recorded pipelines replayed side by side on the same question; (b) the reproducibility page with the bundle checksum and the verified frozen sources.

Discussion

Key findings

Four findings stand out. First, no adaptation of a published method improved answer quality with statistical support in any component; the elaborate strategies matched the answers of simple controls at higher cost. Second, the rule router chose sources significantly better than querying everything and significantly more cheaply than the learned and agentic controllers. Third, ranking every study jointly on quality and cost produced one winner per component, and their combination is exactly the integrated system that the frozen rule had selected before the ranking existed. Fourth, the binding constraint lies upstream: what the retrievers fail to return, no downstream strategy can recover.

Why the simplest strategies win

Three effects combine. First, answers did not separate the strategies: in RQ1 no difference was significant (and, as shown below, none could be), and in RQ2–RQ4 every strategy returned identical answers, so cost and evidence selection decided the ranking. Second, the elaborate methods act after retrieval: rerankers, Steiner trees, diffusion, entity alignment, model-based selection, per-source reasoning, trained routers, calibrated evaluators and agents all reorganise evidence that the retrievers already returned and cannot recover correct papers that were never retrieved, yet they add model calls, tokens and time. Third, the controls are well matched to the questions: the rule router benefits from cue words in templated questions, and the fixed top-K follows the retriever's own ranking. These findings concern this corpus, these retrievers and this model; they do not show that the elaborate mechanisms are ineffective in general.

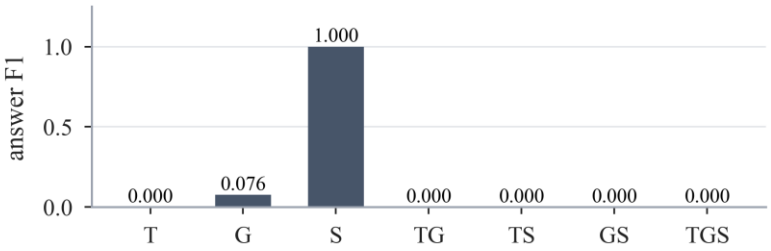
Where answer quality is lost

Fig. 34 locates the loss before generation. In the retrieval-only check, the structured retriever found every correct paper of structured-only questions and a third of those of three-source questions, the graph retriever found 4% of the correct papers of graph-only questions, and the dense text retriever found almost none (0.003 on text-only questions); no retriever found a correct paper of any text-and-graph question. Consequently, in RQ3 every policy answered the structured-only questions exactly

(F1 1.000), the graph-only questions weakly (0.076, since a top-5 holds few of their many correct papers) and no other combination at all; the overall F1 of 0.154 is exactly this mixture. The same pattern bounds RQ1 (gold coverage of at most 0.338) and RQ2 (16 unsupportable bundles). Even where the correct paper was present, the model abstained when it could not verify every condition, which is the behaviour the grounding prompt requests.

	T	G	S	TG	TS	GS	TGS
B1 text	0.003	0	0	0	0	0	0
B2 graph	0	0.040	0	0	0	0.008	0
B3 struct.	0	0	1.000	0	0.004	0.003	0.333
B4 all	0.003	0.040	1.000	0	0.004	0.010	0.333

(a) retrieval-only gold coverage by sources needed



(b) RQ3 answer F1 by sources needed (all policies)

Fig. 34. Where answer quality is lost: (a) retrieval-only gold coverage of the four baselines by the sources a question needs (56 validation questions); (b) answer F1 of every RQ3 policy by the sources a question needs (4 questions per group).

When another strategy is preferable

The winners are the best overall choice, but in specific situations another strategy measured better (Table 18). These group-level differences are descriptive (4–8 questions per group) and were not tested.

Study	Situation	Pick	Evidence
RQ1	answer quality matters more than cost	CM-01	F1 0.283, MRR 0.359; 3.5× tokens
RQ1	every correct paper must reach the model	CM-03	GC 0.338; slowest (40.7 s)
RQ1	speed is the priority	CM-02	6.5 s; lowest MRR
RQ2	a correct paper should rank early	CF-03	MRR 0.063 vs. 0.029; 3.75 calls
RQ2	exactly two sources needed	CF-02	fewest tokens on TG, TS and GS bundles
RQ3	each search is expensive	AR-03	WR 0.012 vs. 0.190
RQ3	text-only questions	AR-03	SF1 1.000 vs. 0.667
RQ3	graph-only questions	AR-01	SF1 1.000 vs. 0.667

Table 18. Situations in which another strategy measured better (descriptive).

Statistical power

The exact sign-flip test cannot return a p-value below $2/2^F$, and Holm multiplies the smallest p-value by the number of pairs k, so the smallest attainable adjusted p-value is $\min(1, 2k/2^F)$ (Fig. 35). With $F = 8$ families the floor is 0.0078, and the smallest attainable adjusted p-values were $10 \times 0.0078 = 0.078$ in RQ1 and $7 \times 0.0078 = 0.055$ in RQ2, both above 0.05: no difference in RQ1 or RQ2 could have been declared significant however consistent it was, and their conclusions rest on the means and on consistency across families. With 14 families, RQ3 and RQ4 could detect differences

and did. Future studies should either use more families or pre-specify fewer pairs; with seven pairs, nine families suffice in principle.

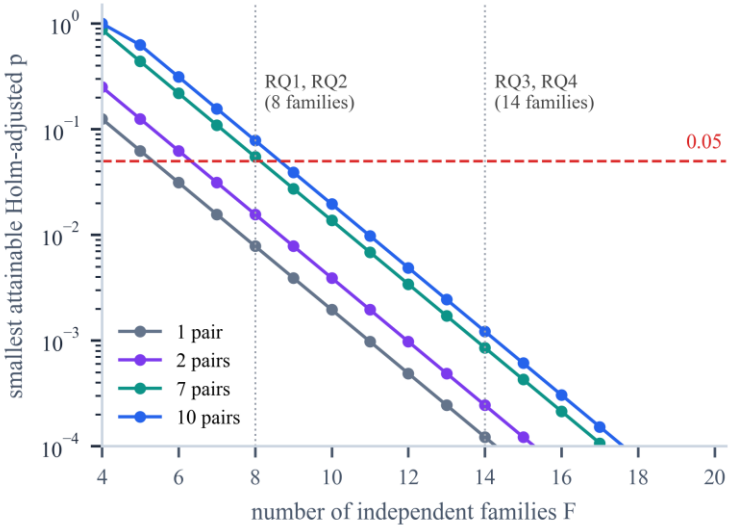


Fig. 35. Smallest attainable Holm-adjusted p-value of the exact sign-flip test as a function of the number of independent families, for 1, 2, 7 and 10 pre-specified pairs. The dashed line marks 0.05.

Computational complexity and scalability

Table 19 summarises the dominant operations of every component together with the measured cost. Three retrievers bound all later work: exact dense search compares the question with all $N = 700{,}244$ normalised 384-dimensional embeddings, about 2.7×10^8 multiply-adds per question; graph traversal is bounded by its caps (at most 20,000 scanned edges), so its cost is independent of the graph's 39.8 million traversal rows; and structured retrieval is a single parameterised query. Context management operates on a candidate pool of at most 256 base nodes and 1,024 base edges, so even super-linear steps are cheap; its measured cost is dominated by embedding the provenance units (CM-01) and by the push budget of the diffusion (CM-03). For fusion and retrieval control the dominant cost is the number of model calls, each of which costs seconds, whereas all non-model computation costs milliseconds; scaling these strategies therefore means scaling model calls, not data.

Component	Dominant operations	Asymptotic cost	Measured cost
B1 dense text	inner products with all paper embeddings, top-5 selection	$O(Nd)$ per question	exact FAISS search over 700,244 papers
B2 graph	seed scan, bounded breadth-first traversal to depth 2	$O(\min(\lvert E\rvert, 2 \times 10^4))$ edges	capped at 2,000 nodes / 20,000 edges
B3 structured	typed plan compiled to one read-only SQL query	one table scan, $O(N)$	one query per question
CM-B0 / CM-B1	serialise records / sort candidates and add parent paths	$O(\lvert V\rvert + \lvert E\rvert)$ / $O(\lvert V\rvert \cdot \log\lvert V\rvert)$	0.1 s / 0.1 s context step; 1 call
CM-01	embed one unit per edge, Kruskal trees, cross-encoder ranking	$O(\lvert E\rvert d + \lvert E\rvert \cdot \log\lvert E\rvert)$	4.2 s context step; 1 call
CM-02	prize assignment, prize-collecting Steiner tree (pcst_fast)	nearly linear in $\lvert E\rvert$	0.0 s context step; 1 call
CM-03	keyword call, push-based diffusion (at most 10^6 pushes), pruning	$O(\sum \text{pushes} \cdot \deg)$	25.2 s context step; 2 calls
CF-B0 / CF-B1 / CF-01	concatenate / section / align and group items	$O(n \cdot \log n)$	1 call
CF-02 / CF-03	selector call / one call per source plus fusion	$O(n)$ + calls	2.00 / 3.75 calls
AR-B0 / AR-B1	all sources / cue matching and routed searches	$O(\lvert q\rvert)$ + searches	3.00 / 2.14 searches; 1 call
AR-01	one T5 forward pass, up to three searches and controller calls	searches + calls	2.71 calls; 24.1 s

Component	Dominant operations	Asymptotic cost	Measured cost
AR-02	cross-encoder scores per item, up to two correction rounds	$O(n)$ scores + searches	2.57 searches; 10.6 s
AR-03	up to four agent, answer, validator and commentor rounds	up to 16 calls	6.54 calls; 45.2 s

Table 19. Computational complexity of the main components and measured cost per question (means). n: candidate items; V and E: nodes and edges of the candidate pool; d: embedding dimension.

Implications for practice

Four guidelines follow. (i) Measure upstream coverage (gold coverage, supportability) before optimising downstream components; no downstream method recovered what retrieval missed. (ii) Treat simple controls as strong baselines and always report cost: a fixed top-K, labelled sections and keyword routing matched the elaborate methods' answers at a fraction of their cost. (iii) Match the retriever to the condition type: exact phrases call for lexical or hybrid text retrieval. (iv) Make experiments replayable: storing every request by its hash made multi-day runs resumable and every number auditable.

Threats to validity

Internal validity is protected by freezing, identical inputs, question-major call order and replay verification; a recorded timing asymmetry slightly favours CM-02 and CM-03 in latency, because their query-specific scores were staged offline while CM-01 computes them inside its timer. *Construct validity*: answer F1 against verified sets is exact, but gold coverage is an upper bound rather than accuracy, and template wording flatters the rule router. *External validity*: one corpus, three simple retrievers and one model; the methods are adaptations, not reproductions. *Statistical conclusion validity*: small cohorts of correlated variants, analysed at the family level as described above.

Conclusion

This work studied, one component at a time, how a retrieval-augmented LLM should use text, graph and structured views of the same scholarly corpus, and built the platform that executes, verifies and replays the study. The main contributions and findings are the following.

- **A controlled platform.** Isolated, identity-joined views of 700,244 papers, a benchmark with verified source necessity, a replay-verified execution protocol and a checksummed interactive application make every reported number traceable to the stored model requests.
- **One winner per component.** Fixed top-K context management (CM-B1), source-separated fusion (CF-B1) and rule-based routing (AR-B1) won on combined quality and cost; no literature-inspired method improved answer quality with statistical support, and the most elaborate controllers cost significantly more.
- **An integrated system chosen before the ranking.** INT-1 = AR-B1 → CM-B1 → CF-B1 answered as well as any single policy (F1 0.154), selected sources significantly better than querying every source (+0.176 source F1) and used 772 more tokens per question than the router alone.
- **The bottleneck is upstream.** Quality is lost before generation, in retrieval coverage, which bounds every downstream strategy.

Limitations. The cohorts are small (8–14 independent families), so the results are exploratory validation results rather than a locked test, and in RQ1 and RQ2 significance was unattainable. Labels are verified by computation; human review of the test sets (2,801 decisions) is outstanding. Text conditions are exact phrases that a meaning-based retriever rarely finds, and template wording favours the rule router. One corpus, one set of simple retrievers and one LLM were used. All literature methods are adaptations: training-heavy components of the originals were replaced or omitted, CM-01 uses graph-native provenance units, and CM-03 converged on few questions under its frozen parameters. Under the frozen top-5 graph retriever, CM-B1 prunes only for policies with several graph calls.

Future work. (i) Raise upstream coverage with lexical or hybrid text retrieval, adaptive graph top-K and structured filters that restrict candidates for the other sources, then repeat the component

studies where answers are possible. (ii) Complete the human review and run the locked test. (iii) Repeat the comparisons with a stronger LLM and larger budgets. (iv) Evaluate fusion on evidence-complete bundles. (v) Increase statistical power with more families and fewer pre-specified pairs. (vi) Test on natural, human-written multi-source questions with verified source necessity.

Data availability

All experiments use the official STaRK-MAG release[36,37], available at https://huggingface.co/datasets/snap-stanford/stark (revision 88269e23e90587f99476c5dd74e235a0877e69be; licence CC-BY-4.0 as recorded on the dataset card). The three views and the benchmark are derived deterministically from this release by the released build scripts and are verified against recorded SHA-256 manifests (benchmark manifest e07f1314…). No human-review decisions are released, and the test and human-test splits were never used.

Code availability

The complete code will be released at https://github.com/[account]/[repository] [link to be added]. It contains the data build, benchmark construction, retrievers, all strategies, the execution protocol, the analyses, the generators of every figure and table in this paper and the interactive application with the recorded runs. Each study is accompanied by its frozen execution plan, stored model requests, run records and analysis files, so every number can be rebuilt without calling the model again.

References

1. Lewis, P. *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* **33**, 9459–9474 (2020).
2. Gao, Y. *et al.* Retrieval-augmented generation for large language models: a survey. Preprint at https://arxiv.org/abs/2312.10997 (2023).
3. He, X. *et al.* G-Retriever: retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems* **37** (2024).
4. Zhu, X., Xie, Y., Liu, Y., Li, Y. & Hu, W. Knowledge graph-guided retrieval augmented generation. Preprint at https://arxiv.org/abs/2502.06864 (2025).
5. Zhou, Z. *et al.* Query-aware flow diffusion for graph-based RAG with retrieval guarantees. In *Proc. International Conference on Learning Representations* (2026).
6. Liang, L. *et al.* KAG: boosting LLMs in professional domains via knowledge augmented generation. Preprint at https://arxiv.org/abs/2409.13731 (2024).
7. Wu, Y. *et al.* SAG: SQL-retrieval augmented generation with query-time dynamic hyperedges. Preprint at https://arxiv.org/abs/2606.15971 (2026).
8. Hsiao, C.-H., Wang, Y.-C., Lin, T.-S., Yeh, Y.-R. & Chen, C.-S. MegaRAG: multimodal knowledge graph-based retrieval augmented generation. Preprint at https://arxiv.org/abs/2512.20626 (2025).
9. Jeong, S., Baek, J., Cho, S., Hwang, S. J. & Park, J. C. Adaptive-RAG: learning to adapt retrieval-augmented large language models through question complexity. In *Proc. 2024 Conference of the North American Chapter of the Association for Computational Linguistics* (2024).
10. Yan, S.-Q., Gu, J.-C., Zhu, Y. & Ling, Z.-H. Corrective retrieval augmented generation. Preprint at https://arxiv.org/abs/2401.15884 (2024).
11. Anonymous. AGENT-G: an agentic framework for graph retrieval augmented generation. Submitted to *International Conference on Learning Representations*; https://openreview.net/forum?id=g2C947jjjQ (2024).
12. Holm, S. A simple sequentially rejective multiple test procedure. *Scand. J. Stat.* **6**, 65–70 (1979).
13. Efron, B. & Tibshirani, R. J. *An Introduction to the Bootstrap* (Chapman & Hall, 1993).
14. Cameron, A. C., Gelbach, J. B. & Miller, D. L. Bootstrap-based improvements for inference with clustered errors. *Rev. Econ. Stat.* **90**, 414–427 (2008).
15. Reimers, N. & Gurevych, I. Sentence-BERT: sentence embeddings using Siamese BERT-networks. In *Proc. 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* 3982–3992 (2019).
16. Wang, W. *et al.* MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems* **33** (2020).
17. Johnson, J., Douze, M. & Jégou, H. Billion-scale similarity search with GPUs. *IEEE Trans. Big Data* **7**, 535–547 (2021).
18. Liu, N. F. *et al.* Lost in the middle: how language models use long contexts. *Trans. Assoc. Comput. Linguist.* **12**, 157–173 (2024).
19. Jiang, H. *et al.* LongLLMLingua: accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics* (2024).

20. Jiang, Z. *et al.* Active retrieval augmented generation. In *Proc. 2023 Conference on Empirical Methods in Natural Language Processing* 7969–7992 (2023).
21. Asai, A., Wu, Z., Wang, Y., Sil, A. & Hajishirzi, H. Self-RAG: learning to retrieve, generate, and critique through self-reflection. In *Proc. International Conference on Learning Representations* (2024).
22. Edge, D. *et al.* From local to global: a graph RAG approach to query-focused summarization. Preprint at https://arxiv.org/abs/2404.16130 (2024).
23. Goemans, M. X. & Williamson, D. P. A general approximation technique for constrained forest problems. *SIAM J. Comput.* **24**, 296–317 (1995).
24. Hegde, C., Indyk, P. & Schmidt, L. A nearly-linear time framework for graph-structured sparsity. In *Proc. 32nd International Conference on Machine Learning* 928–937 (2015).
25. Fountoulakis, K., Wang, D. & Yang, S. p-Norm flow diffusion for local graph clustering. In *Proc. 37th International Conference on Machine Learning* 3222–3232 (2020).
26. Sun, J. *et al.* Think-on-Graph: deep and responsible reasoning of large language model on knowledge graph. In *Proc. International Conference on Learning Representations* (2024).
27. Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M. & Su, Y. HippoRAG: neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems* **37** (2024).
28. Peng, B. *et al.* Graph retrieval-augmented generation: a survey. Preprint at https://arxiv.org/abs/2408.08921 (2024).
29. Oguz, B. *et al.* UniK-QA: unified representations of structured and unstructured knowledge for open-domain question answering. In *Findings of the Association for Computational Linguistics: NAACL 2022* 1535–1546 (2022).
30. Jiang, J. *et al.* StructGPT: a general framework for large language model to reason over structured data. In *Proc. 2023 Conference on Empirical Methods in Natural Language Processing* 9237–9251 (2023).
31. Sarmah, B. *et al.* HybridRAG: integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. In *Proc. 5th ACM International Conference on AI in Finance* (2024).
32. Bu, C. *et al.* Query-driven multimodal GraphRAG: dynamic local knowledge graph construction for online reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025* 21360–21380 (2025).
33. Trivedi, H., Balasubramanian, N., Khot, T. & Sabharwal, A. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proc. 61st Annual Meeting of the Association for Computational Linguistics* 10014–10037 (2023).
34. Yao, S. *et al.* ReAct: synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations* (2023).
35. Maharjan, A., Kaur, D. & Molyet, R. G. Agent-orchestrated adaptive RAG: a comparative study on structured and multi-hop retrieval. Preprint at https://arxiv.org/abs/2606.05658 (2026).
36. Wu, S. *et al.* STaRK: benchmarking LLM retrieval on textual and relational knowledge bases. In *Advances in Neural Information Processing Systems* **37**, Datasets and Benchmarks Track (2024).
37. Wang, K. *et al.* Microsoft Academic Graph: when experts are not enough. *Quant. Sci. Stud.* **1**, 396–413 (2020).
38. Dror, R., Baumer, G., Shlomov, S. & Reichart, R. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proc. 56th Annual Meeting of the Association for Computational Linguistics* 1383–1392 (2018).
39. Good, P. *Permutation, Parametric, and Bootstrap Tests of Hypotheses* 3rd edn (Springer, 2005).
40. Raasveldt, M. & Mühleisen, H. DuckDB: an embeddable analytical database. In *Proc. 2019 International Conference on Management of Data* 1981–1984 (2019).
41. OpenAI. gpt-oss-120b & gpt-oss-20b model card. Preprint at https://arxiv.org/abs/2508.10925 (2025).
42. Ollama. Ollama: get up and running with large language models locally. https://ollama.com (2025).
43. Kruskal, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proc. Am. Math. Soc.* **7**, 48–50 (1956).
44. Xiao, S. *et al.* C-Pack: packed resources for general Chinese embeddings. In *Proc. 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024).
45. Raffel, C. *et al.* Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**, 1–67 (2020).
46. Nogueira, R. & Cho, K. Passage re-ranking with BERT. Preprint at https://arxiv.org/abs/1901.04085 (2019).
47. Bajaj, P. *et al.* MS MARCO: a human generated machine reading comprehension dataset. Preprint at https://arxiv.org/abs/1611.09268 (2016).
48. Platt, J. C. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers* (eds Smola, A. J., Bartlett, P., Schölkopf, B. & Schuurmans, D.) 61–74 (MIT Press, 1999).

Acknowledgements

[To be completed by the author.]

Author contributions

A.B. conceived the study, designed and implemented the system and the experiments, analysed the results and wrote the manuscript. [To be confirmed by the author.]

Funding

[To be completed by the author.]

Declarations

Competing interests

[To be completed by the author; for example: "The author declares no competing interests."]

Ethical approval

Not applicable: the study uses a publicly released bibliographic dataset and involved no human participants. [To be confirmed by the author.]

Additional information

Correspondence and requests for materials should be addressed to A.B.